KOCAELİ ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BİTİRME PROJESİ

No-Code Backend Servisi Platformu

MUHAMMET ALİ BAKINÇ
SAMET TOPRAK

KOCAELİ 2023

İçindekiler

1. Giriş 6
1.1. Problemin Tanımı 6

2. Kaynak Araştırması 7
2.1. Low-Code / No-Code (LC/NC) Platformlar 7
2.2. Uygulama Geliştirme Ortamları 8
IntelliJ IDEA 8
DataGrip 8
Visual Studio Code 9
2.3. Veritabanı Yönetim Sistemleri (VYS) 9
Ağ Veri Tabanı Yönetim Sistemi 10
Hiyerarşik Veri Tabanı Yönetim Sistemi 10
İlişkisel Veri Tabanı Yönetim Sistemi 10
Nesneye Dayalı Veri Tabanı Yönetim Sistemi 11
2.4. LC/NC VYS Sistemlerin Genel/Temel Özellikleri 11
2.5. Benzer Sistemler / Özellik Karşılaştırma Tablosu 12

3. Tanımlar / Teknolojiler 13
3.1. QueryBuilder 13
3.2. HTTP / HTTPS 13
HTTPS 13
3.3. API / RestFul API 14
API 14
REST Nedir? 14
RESTful API İçin Gereken Özellikler 14
3.4. Docker 14
Docker 14
3.5. Mikro Servis Mimarisi 15
3.6. MVC Tasarım Deseni 16

4. Çözüm Önerisi / Yöntem 17
4.1. Sistem Gereksinim Analizi 17
4.2. Sistem Tasarımı 17
4.2.1. Genel Mimari / Şematik 17
4.2.2. Varlık-Bağıntı (ER) Modeli 17
4.2.3. Kullanım Senaryoları (UML) 18
4.2.4. Sınıf Diyagramı (UML) 19
4.2.5. Prototipleme (Wiframing) 20
4.3. Kullanılacak Teknolojiler 22

4.3.1. Backend 22
Java 22
Spring Framework 22
Spring Boot 23
4.3.2. Frontend 24
HTML 24
CSS 25
Bootstrap 25
Javascript 26
React 27
4.3.3. Servisler 28
PostgreSQL 28

5. Sonuç / Tartışma 30

ÖZET

Bu çalışmanın amacı Low-Code No-Code bir sistem oluşturarak kullanıcıların yapmak istedikleri bir projede veri tabanlarını olabildiğince az kod yazarak oluşturmasına yardım etmek ve oluşturdukları bu veri tabanlarını yönetmesini sağlamaktır.

İlk olarak benzer sistemlerin özellikleri tanımlanmıştır. Diğer Low-Code No-Code sistemler arasındaki farklar araştırılarak sistemlerinde nasıl bir yol izlendiği bulunmuştur. Benzer sistemlerin özellikleri bir tabloda karşılaştırılmıştır. Yapacağımız projede amacımız veri tabanlarını yönetmek olduğu için buna yardımcı olacak sistemlerin neler olduğu tespit edilip daha sonra tanımlamaları yapılmıştır.

Bizim yapmak istediğimiz sistemin tanımı yapıldıktan sonra alttaki diyagramlar ile tasarladığımız sistemin bir özeti gösterilmiştir. Sistemde esin kaynağı olarak Spring Framework'ün çalışma mantığı alınmıştır.

ABSTRACT

The aim of this study is to create a Low-Code No-Code system, to help users create databases by writing as little code as possible in a project they want to do and to enable them to manage these databases.

First, the properties of similar systems are defined. The differences between other Low-Code and No-Code systems were investigated and it was found out how the system was followed. The characteristics of similar systems are compared in a table. Since our aim in the project we will do is to manage databases, the systems that will help this have been determined and then their definitions have been made.

After the definition of the system we want to build, a summary of the system we designed is shown with the diagrams below. The working logic of the Spring Framework was taken as the inspiration for the system.

1. GİRİŞ

1.1. Problemin Tanımı

Kullanıcıların veri tabanlarını kontrol etmesi projelerin ölçeğinden dolayı giderek zorlaşmaktadır. Ayrıca yazılıma başlama zorluğunun oldukça azalmasının ardından yeni oluşturulmak istenen projelerin veri tabanlarını yönetmek için daha kolay yolların bulunması da bir ihtiyaç haline gelmiş durumda. Biz de teknik bilgisi olmayan kullanıcıların oluşturmak projeler için veri tabanlarını yönetmesini kolaylaştırmanın yollarını aramak için bu çalışmayı yapmaya karar verdik.

2. Kaynak Araştırması

2.1. Low-Code / No-Code (LC/NC) Platformlar

Low–Code Platformlar:

Backendless

- SQL ve NoSQL veri tabanları
- Cloud code
- Codeless
- User-management
- Geolocation

Heroku

- Cloud Computing
- Birden fazla dil seçeneği
- Mobil geliştirme
- Web uygulaması geliştirme
- GUI
- Auto scaling

Mendix

- Open model API
- Authentication
- Pre-built templates
- Real-time monitoring
- Drag and drop interface

Zoho Creator

- Drag and drop interface
- Market place and developer community
- Web mobil ve native mobil erişimi
- Real-time reporting
- Dosya saklama

No-Code Platformlar:

Airtable

- Veri saklama ve dağıtımını yönetme
- Drag and drop interface
- E-mail response control
- Built-in database
- Automatic backup
- Advertising management

Bubble

- Design templates
- Drag and drop interface

- Workflow management
- Automatic backups
- Veri ve hesap yönetimi
- JavaScript ve API entegrasyonu

Voiceflow

- Alexa ve Google için sesli komut uygulamalarında kullanılır
- Drag and drop blocks
- Machine learning
- One-click publish

Backendless

- API service
- IU Builder
- Document management
- Debugging
- Push notifications
- Database support

2.2. Uygulama Geliştirme Ortamları

Projede geliştirme yapabilmek için seçtiğimiz geliştirme ortamlarının kullanımı kolay ve gelişmiş özelliklere sahip olmasını önceliklendirdik. Bunun için Java – Spring Boot geliştirmek için Intellij Idea IDE'sini seçtik. Veritabanı yönetimi için yine aynı firmanın ürünü olan DataGrip kullanmaya karar verdik. Web ön yüzü de Visual Studio Code tercih ettik. Kısaca ürünler hakkında bilgi verip, seçme nedenlerimizi açıklayalım.

IntelliJ IDEA

IntelliJ IDEA, bir tümleşik geliştirme ortamı (IDE) olarak Java yazılım geliştirme sürecinde kullanılan bir araçtır. JetBrains tarafından geliştirilen IntelliJ, hem ticari hem de topluluk sürümleriyle kullanıcıların ihtiyaçlarına yönelik çözümler sunar. Ticari sürümüyle birlikte gelişmiş özellikler sunan IntelliJ IDEA'nın Apache 2 lisansı altında yayınlanan topluluk sürümü de mevcuttur. Her iki sürüm de ticari amaçlı kullanıma uygundur. İlk olarak 2001 yılında piyasaya sürülen IntelliJ IDEA, Java IDE'leri arasında öne çıkan bir yazılımdır ve kod gezinme ve yeniden düzenleme gibi gelişmiş özelliklere sahiptir. 2010 yılında InfoWorld tarafından yapılan bir karşılaştırmada, Eclipse, IntelliJ IDEA, NetBeans ve JDeveloper IDE'leri arasında yapılan testlerde en yüksek puanı alarak başarısını kanıtlamıştır. Hem yüksek performanslı olması hem de Java'ya en uyumlu ve en çok tercih edilen IDE'lerin başında gelmesi nedeniyle kullanmayı tercih ettik. Spring Boot'a özel eklentileriyle kullanımı hayli kolaylaştırmaktadır.

DataGrip

DataGrip yine IntelliJ üretimi olan çok motorlu bir veritabanı ortamıdır. Sade ve anlaşılır arayüzü ile keyifli bir kullanım sunmaktadır. MySQL, PostgreSQL, Microsoft SQL Server, Oracle, Sybase, DB2, SQLite, HyperSQL, Apache Derby ve H2 gibi birçok veritabanı sistemini destekler. Otomatik tamamlama, analiz ve navigasyon gibi SQL kodlarını düzenlemek için akıllı kodlama yardımı sağlayan bir editör içerir. Ayrıca sorguları çalıştırmak ve profil oluşturmak için bir sorgu konsolu da sunar. Biz projemizde veritabanı olarak PostgreSQL kullanıyor olacağız.

Bunun için ikinci bir tercih olarak PostgreSQL'in kendi uygulaması olan pgAdmin4 de tercih edilebilirdi. Ancak pgAdmin4'e kıyasla çok daha anlaşılabilir ekranlara sahip olması ve kullanıcı dostu birçok kolaylaştırıcı desteğe sahip olması bu uygulamayı tercih için çok etkili olmuştur.

Visual Studio Code

Microsoft tarafından geliştirilen Visual Studio Code, Windows, Linux ve MacOS için tasarlanmış bir kaynak kodu düzenleyicisidir. Bu düzenleyici, birçok özellik içerir ve geliştiricilere kod yazma deneyiminde kolaylık sağlar. Hata ayıklama yetenekleri, entegre Git kontrolü, sözdizimi vurgulama, akıllı kod tamamlama, snippet'ler ve kod yeniden düzenleme gibi özellikleri bulunur. Ayrıca, kullanıcılar tarafından özelleştirilebilir, böylece editörün teması, klavye kısayolları ve tercihler gibi özellikler kişiselleştirilebilir. Visual Studio Code resmi olarak tescilli bir lisans altında sunulmasına rağmen, ücretsiz ve açık kaynaklı bir yazılımdır. Bu nedenle, kullanıcılar ücretsiz olarak indirip kullanabilir ve kaynak kodunu istedikleri gibi inceleyebilirler. Yine ön yüz için kullanacağımız React için en uyumlu IDE'lerin başında gelmesi, çok fazla tercih edilmesi ve gelişmiş kodlama özelliklerine sahip olması seçim nedenimiz olmuştur.

2.3. Veritabanı Yönetim Sistemleri (VYS)

Veritabanı yönetim sistemi (VYS), yeni veritabanlarının oluşturulması, varolan veritabanlarının düzenlenmesi, yönetilmesi, geliştirilmesi ve belirli amaçlar için kullanılması için kullanılan bir yazılımdır. Ayrıca, veritabanlarının bakımı da VYS tarafından gerçekleştirilir. Veritabanı yönetim sistemleri, verilerin tutulduğu yapıları organize etmek, verilere erişimi sağlamak, sorguları yürütmek, güvenlik ayarlarını yönetmek ve performans optimizasyonunu sağlamak gibi işlevleri yerine getirir.

VYS'ler, veritabanı oluşturma ve yapılandırma, tablo ve indeks oluşturma, veri ekleme, düzenleme ve silme, veri sorgulama ve raporlama gibi temel veritabanı işlemlerini gerçekleştirmek için kullanılır. Ayrıca, kullanıcıların veritabanlarına erişim yetkilerini yönetmek, veritabanı yedeklemelerini oluşturmak ve geri yüklemek, veritabanı performansını izlemek ve optimize etmek gibi gelişmiş özelliklere de sahip olabilirler.

Bazı popüler veritabanı yönetim sistemleri arasında MySQL, Oracle, Microsoft SQL Server, PostgreSQL ve MongoDB gibi farklı tiplerde veritabanlarını destekleyen çeşitli yazılımlar bulunmaktadır. Bu sistemler, farklı gereksinimlere ve kullanım senaryolarına uyacak şekilde tasarlanmıştır ve geniş bir yelpazede kullanılmaktadır. VYS; veri tanımlama, veri alma ve kullanıcı yönetimini içerir. Veri tanımı, bir veritabanındaki verilerin organizasyonunu tanımlayan tanımların oluşturulması, değiştirilmesi ve kaldırılmasıdır. Veri alma, uygulamalar tarafından çeşitli amaçlarla kullanılabilecek veritabanından verilerin alınmasıdır. Kullanıcı yönetimi; performansın izlenmesi, veri bütünlüğünün korunması, eşzamanlılık kontrolü ile uğraşılması ve beklenmeyen bir arıza nedeniyle bozulan bilgilerin kurtarılması yoluyla veri güvenliğinin uygulanmasını içerir. Ayrıca, kullanıcıların kaydedilmesini ve izlenmesini de içerir.

Bir VYS üç öğeyi yönetir; fiziksel veri tabanı, veri tabanı motoru ve veri tabanı şeması. Fiziksel veritabanı, veri içeren bir dosya koleksiyonudur. Veri tabanı motoru, verilere erişilmesini, değiştirilmesini ve kilitlenmesini sağlayan yazılımdır. Veri tabanı şeması, veri tabanındaki mantıksal yapının özelliğidir. Bu üç unsur; güvenlik, veri bütünlüğü, eşzamanlılık ve tek tip veri yönetim prosedürü sağlar. VYS; yedekleme ve kurtarma, değişiklik yönetimi ve performans izleme gibi yönetim görevlerini destekler. Ayrıca, yeniden başlatma, geri alma, kurtarma ve ayrıca etkinlik veri tabanı etkinliğini denetlemekten de sorumludur.

Ağ Veri Tabanı Yönetim Sistemi

Bir ağ veri tabanı, her bir veri noktasının birden çok birincil ve ikincil veri noktasına bağlı olduğu bir veri topluluğudur. Birbirine bağlı veri noktalarına sahip olmak, bu veri modelini kullanım açısından daha esnek hale getirir. Bir ağ veri tabanı yönetim sistemi, birbiriyle ilişkili veri noktalarını bir ağ veri tabanında depolayan ve yöneten bir yazılımdır. Bu yazılım, yukarıdan aşağıya akışın yanı sıra veri noktaları arasında ara bağlantıya izin vermeyen hiyerarşik bir veri tabanı modelinin eksikliklerini gidermek için oluşturulmuştur.

Bir ağ veri tabanı sistemi tipik olarak: hızlı veri erişimini kolaylaştırır; çoktan çoğa ilişkisel veri tabanı modellerini destekler; karmaşık veri tabanı yapılarının oluşturulmasına ve yönetilmesine izin verir.

Hiyerarşik Veri Tabanı Yönetim Sistemi

Hiyerarşik bir veri tabanı, depolanan verilerin bağlantılar yoluyla bağlandığı ve yukarıdan aşağıya doğru düzenlendiği ağaç benzeri bir yapı halinde düzenlenen bir veri topluluğudur. Birincil veri noktası en üsttedir ve ikincil veri noktaları alakalarına bağlı olarak hiyerarşide onu takip eder. İşletmenizin organizasyon yapısı, hiyerarşik bir veri tabanının mükemmel bir örneğidir.

Hiyerarşik bir veri tabanı yönetim sistemi, hiyerarşik veri tabanlarını depolayan ve yöneten bir yazılımdır. İş süreçlerindeki kullanıma bağlı olarak veri hiyerarşisinde veya akışında doğruluğu korur. Hiyerarşik bir sistem içindeki veriler tipik olarak: eklemek ve silmek kolaydır; aramak ve almak kolaydır; bire çok ilişkisel veri modelini takip eder.

İlişkisel Veri Tabanı Yönetim Sistemi

İlişkisel veri tabanı, daha iyi kullanılabilirlik için farklı veri noktalarının birleştirilebilmesi için birbiriyle ilişkili bir veri topluluğudur. İlgili noktalar zaman, veri veya mantık olabilir ve ilişki aşağıdaki şekillerde kategorize edilebilir:

Bire bir: Bir tablodaki bir veri noktası, başka bir tablodaki bir veri noktasıyla ilişkilidir.

Birden çoğa: Bir tablodaki bir veri noktası, başka bir tablodaki birden çok veri noktasıyla ilişkilidir.

Çoktan bire: Bir tablodaki birden çok veri noktası, başka bir tablodaki bir veri noktasıyla ilişkilidir.

Çoktan çoğa: Bir tablodaki birden çok veri noktası, başka bir tablodaki birden çok veri noktasıyla ilişkilidir.

İlişkisel veri tabanı yönetim sistemi, ilişkisel veri tabanlarının depolanmasını ve paylaşılabilirliğini yöneten bir yazılımdır. Birden çok veri noktası arasında işlevsel bağımlılıklar oluşturarak verileri ilişkisel bir veri tabanında düzenler. Ayrıca verileri düzenli bir şekilde depolar. Böylece çalışanların günlük görevleri için verileri bulmasını ve kullanmasını kolaylaştırır. İlişkisel bir veri yapısı, çalışanların sorguları çalıştırmasına ve ihtiyaç duydukları bilgileri bulmasına izin vermek için yapılandırılmış sorgu dilini (SQL) kullanır. İlişkisel bir veri tabanı yönetim sistemi tipik olarak: büyük miktarda veri depolar; hızlı veri alımını sağlar; kullanıcıların aynı anda birden çok veri öğesine erişmesine izin verin.

Nesneye Dayalı Veri Tabanı Yönetim Sistemi

Nesneye dayalı bir veri tabanı, bir nesne biçiminde sunulan bir veri topluluğudur. Birden çok veri noktası tek bir birim veya nesnede birleştirilir ve çalışanların verileri bulmasını ve kullanmasını kolaylaştırır. Bu tür veri tabanı, yazılım geliştirme ve programlama gibi daha hızlı karar vermeyi gerektiren yüksek performanslı görevleri gerçekleştirmek için kullanılır.

Nesneye dayalı veri tabanı yönetim sistemi, veri tabanlarını nesneler olarak depolayan ve yöneten bir yazılımdır. Çalışanların bireysel veri noktaları yerine eksiksiz nesneleri aramasına olanak tanıyarak daha hızlı arama sağlar. Nesneye dayalı bir veri tabanı yapısı tipik olarak: nesnelerin amaçlarını kaybetmemeleri için veri tabanı nesneleri ile gerçek dünya senaryoları arasında doğrudan bir ilişki sürdürür; çalışanların nesneleri hızlı bir şekilde bulmaları ve kullanmaları için bir nesne tanımlayıcısı sağlar; resimler, metinler ve grafikler gibi farklı veri türlerini işler.

2.4. LC/NC VYS Sistemlerin Genel/Temel Özellikleri

Low-Code No-Code platformlar, geliştirme gerektirmeyen ya da çok az kod yazılması gerektiren platformlardır. Bu platformlar, kullanıcıların çeşitli uygulamalar oluşturmak için drag-and-drop (sürükle-bırak) özelliği gibi araçlar sunarlar. Bu uygulamalar genellikle veri tabanı yönetimi, form oluşturma veya raporlama gibi işlemleri gerçekleştirirler.

Veri yönetim sistemleri verilerin toplanması, depolanması, düzenlenmesi ve kullanılması gibi işlemleri gerçekleştiren yazılımlardır. Low-Code, No-Code platformlarında, veri yönetim sistemleri genellikle aşağıdaki özellikleri sunar:

Veri tabanı oluşturma ve yönetme:

Kullanıcılar, veri tabanı oluşturmak ve verileri eklemek, güncellemek ve silmek için kullanabilecekleri arayüzler sunulur.

Veri Import/Export:

Kullanıcılar, verilerini başka sistemlerden alıp, Low-Code No-Code platformlarına aktarmak veya platformdaki verileri başka sistemlere aktarmak için kullanabilecekleri arayüzler sunulur.

Veri filtreleme ve sorgulama:

Kullanıcılar, verilerini filtreleyebilir ve özel sorgular oluşturarak istedikleri verilere ulaşabilirler.

Veri güvenliği:

Veri yönetim sistemleri, verilerin güvenliğini sağlamak için gerekli olan önlemleri alır. Örneğin, verilerin şifrelenmesi ve veritabanı güvenliğinin sağlanması gibi önlemler alınır.

Kullanım kolaylığı:

Low-Code, No-Code platformları genellikle kullanımı kolay olan arayüzler sunarlar, böylece kullanıcılar veri yönetim sistemlerini kolayca kullanabilirler.

2.5. Benzer Sistemler / Özellik Karşılaştırma Tablosu

	Bubble	Backendless	Xano
1-DEVELOP			
1.1-Database	✓	✓	✓
1.1.1-Create New Table	✓	✓	✓
1.1.2-Create New Row	✓	✓	✓
1.1.3-Delete Table	✓	✓	✓
1.1.4-Delete Row	✓	✓	✓
1.1.5-Data Modify	×	×	×
1.1.6-Data Update	✓	✓	✓
1.1.6-Connect own SQL database	✓	×	×
1.1.7-Connect own REST API	✓	×	✓
1.2-Export and manage data	✓	✓	✓
1.2.1-CSV Export	✓	✓	✓
1.2.2-Excel Export	✓	✓	✓
1.3-User Management	✓	✓	✓
1.3.1-Sign the user up	✓	✓	✓
1.3.2-Log the user in	✓	✓	✓
1.3.3-Sign and Log with the social network	✓	✓	✓
1.3.4-Log the user out	✓	✓	✓
1.3.5-Send Confirmation e-mail	✓	✓	✓
1.3.6-Send password reset e-mail	✓	✓	✓
1.3.7-Check password for the current user	✓	✓	✓
1.3.8-Assign a temp password to a user	✓	✓	✓
1.3.9-Connect own Firebase Authentication	×	×	×
1.4-Geolocation	×	✓	✓
1.5-Security	✓	✓	✓
1.6-Logging	✓	✓	×
1.7-Real-Time Data	×	✓	×
1.8-Pub-Sub Messaging	×	✓	×
1.9-Test-Debug	×	×	✓
1.10-Media Upload	×	×	✓

3. Tanımlar / Teknolojiler

3.1. QueryBuilder

Query Builder, veritabanı sorgularının oluşturulması için kullanılan bir araçtır. Bu araç, veritabanı sorgularını yaratırken kod yazmak zorunda olmadan, grafik arayüzünden sorgular oluşturmamıza olanak tanır. Query Builder'lar, çeşitli veritabanı yönetim sistemleri (DBMS) için mevcuttur ve sıklıkla web uygulamalarında kullanılır.

Query Builder'lar, veritabanı sorgularını oluştururken kullanılan yapıları (örneğin SELECT, FROM, WHERE gibi) düzenli bir şekilde gösterirler ve bu yapıları kullanarak sorgular oluşturmayı kolaylaştırırlar. Bu sayede veritabanı sorgularının oluşturulması için gerekli olan teknik bilgiye sahip olmaya gerek kalmaz. Bunun yerine, Query Builder arayüzünde görsel olarak sorgu oluşturulabilir. Query Builder kullanarak veri tabanındaki objeleri arama ve filtreleme, obje ve sütunları seçme, objeler arasında ilişkiler kurma ve sorgulama gibi özellikleri hiç SQL bilgisine sahip olmadan da gerçekleştirebiliriz.

3.2. HTTP / HTTPS

HTTP (Hypertext Transfer Protocol), bir ağ üzerinden metin, resimler, video ve diğer verilerin aktarılması için kullanılan bir protokoldür. Bu protokol, web tarayıcıları (örneğin Chrome, Firefox, Safari) ve web sunucuları (örneğin Apache, Nginx, IIS) arasında bir iletişim kurulmasını ve verilerin aktarılmasını sağlar.

İstemci, bir web sayfasını görüntülemek istediğinde, tarayıcısı aracılığıyla sunucuya bir HTTP isteği gönderir. Bu istek, sunucunun istenilen sayfayı göndermesini ister. Sunucu, isteği alır ve istenilen sayfayı tarayıcıya gönderir. Tarayıcı, sayfayı aldıktan sonra sayfayı görüntüler.

HTTP, bir web sayfasının sadece bir kez indirilmesi için kullanılır. Bu nedenle, tarayıcınız bir web sayfasını görüntülediğinizde, tarayıcı sayfayı önbellekte (cache) saklar. Bu sayede, bir sonraki ziyaretinizde sayfa daha hızlı yüklenebilir. Eğer tarayıcı önbelleğinde sayfa yoksa veya sayfa güncellenmiş ise, tarayıcı sunucuya bir HTTP isteği gönderir ve güncel sayfayı indirir. HTTP bir taşıma katmanı protokolüdür.

HTTPS

HTTPS (Hypertext Transfer Protocol Secure), verilerin güvenliğini sağlamak için kullanılan bir internet protokolüdür. HTTPS, verilerin internet üzerinden iletimi sırasında şifrelenmesini sağlar ve bu sayede verilerin güvenliği korunur. HTTPS, genellikle web tarayıcıları ve web sunucuları arasında veri aktarımı için kullanılır ve özellikle güvenli veri aktarımı gerektiren uygulamalar için tercih edilir.

HTTPS, HTTP protokolünün güvenliğini sağlamak için kullanılan bir ekstra katmandır. Bu katman, SSL (Secure Sockets Layer) veya TLS (Transport Layer Security) gibi güvenlik protokolleri kullanılarak verilerin şifrelenmesini sağlar. Bu sayede, veriler internet üzerinden iletildiğinde güvenliği korunur ve verilerin gizliliği sağlanır.

3.3. API / RestFul API

API

Bir API (application programming interface) yazılım sistemleriyle iletişim kurmak için takip edilmesi gereken kuralları tanımlar. API, bir sistemin işlevlerini ve verilerini başka bir sisteme açık hale getirir ve bu sayede iki sistem arasında veri akışı sağlar. Bu, sistemler arasında entegrasyonu ve veri paylaşımını kolaylaştırır.

Geliştiriciler kendi geliştirdikleri yazılımlar için bir API oluşturarak diğer uygulamaların kendi uygulamalarıyla iletişim kurmasını sağlarlar. Örneğin, bir web uygulaması, bir veritabanı sistemine erişmek için veritabanı sisteminin API'sini kullanabilir. Aynı şekilde, bir mobil uygulama, bir web servisinin işlevlerini kullanmak için web servisinin API'sini kullanabilir.

API'ler, genellikle bir programlama diline (örneğin Java, Python, C#) göre tasarlandıklarından, API'leri kullanmak için bu dillerde programlama bilgisi gerekir. API'ler, genellikle bir dokümantasyon veya API kılavuzu ile birlikte sunulurlar ve bu dokümantasyonda, API'lerin nasıl kullanılacağı ve hangi işlevlerin kullanılabileceği gibi bilgiler verilir.

API'ler, yazılım sistemlerinin etkileşimini kolaylaştırmakla birlikte, aynı zamanda sistemler arasında veri paylaşımını da sağlar. Bu sayede, farklı sistemler arasında veri toplama, işleme ve paylaşma gibi işlemler yapılabilir.

REST Nedir?

REST (Representational State Transfer) bir mimari kısıtlamalar bütünüdür. REST bir protokol veya standart değildir. REST ilk olarak internet üzerindeki iletişimi yönetmesi için oluşturulmuş olan bir yapıdır. REST mimarisi geniş ölçekte yüksek performans ve güvenilir iletişimi sağlamak için kullanılabilir.

RESTful API İçin Gereken Özellikler

- RESTful API isteklerin HTTP üzerinden yönetildiği bir istemci-sunucu mimarisi olmalıdır.
- Durumsuz (stateless)
- Önbellek kullanabilir (cacheable)
- Uniform interface kullanılmalı bu sayede iletişim tek bir standart formda olabilir.
- Katmanlı sistem

3.4. Docker

Docker

Docker, bir uygulamanın çalıştırılması için gerekli olan tüm kaynakları (örneğin, kod, ortam değişkenleri, kütüphaneler vb.) bir arada tutan bir konteyner teknolojisi kullanır. Bu sayede uygulama farklı ortamlarda ve makinelere kolayca dağıtılabilir ve çalıştırılabilir. Docker

ayrıca zaman ve kaynak tasarrufu sağlar ve uygulamaların test edilmesine ve yayınlanmasına yardımcı olur. Docker birçok farklı işletim sistemi ve platformda kullanılabilir ve çeşitli programlama dilleriyle uyumludur. Docker, uygulamaların çalıştırılması için gereken tüm gereksinimleri bir arada toplayarak, bu gereksinimleri bir "konteyner" olarak adlandırdığı bir paket haline getirir. Bu konteynerler, uygulamanın çalıştığı bilgisayarın özelliklerine bağlı olmadan, farklı bir bilgisayarda da çalıştırılabilir. Böylece, bir uygulamanın geliştirilmesi ve test edilmesi sırasında karşılaşılabilecek problemler azaltılır ve uygulamanın dağıtımı kolaylaşır.

Docker, uygulamaların çalıştırılması sırasında oluşabilecek sorunları önleyerek zaman ve kaynak tasarrufu sağlar. Örneğin, bir uygulama bir makine üzerinde çalıştığında, uygulamanın ihtiyaç duyduğu tüm kütüphanelerin ve ortam değişkenlerinin yüklenmesi gerekir. Ancak Docker kullanılarak, uygulamanın ihtiyaç duyduğu tüm bu öğeler bir konteyner içinde toplanabilir ve bu konteyner farklı makinelere kolayca dağıtılabilir. Bu sayede uygulama, farklı ortamlarda ve farklı makinelere kolayca çalıştırılabilir.

Docker, ayrıca uygulamaların test edilmesi ve yayınlanması için de kullanılabilir. Örneğin, bir uygulama geliştiricisi bir uygulama üzerinde çalışırken, uygulamanın çalıştığı ortamı tasarımdan farklı olabilir. Bu durumda, Docker kullanılarak geliştiricinin kendi bilgisayarında aynı ortamda uygulamanın çalıştırılması sağlanabilir, böylece uygulama üzerinde yapılan değişikliklerin doğru şekilde çalışıp çalışmadığı test edilebilir.

Docker, aynı zamanda bir dağıtım platformu olarak da kullanılabilir. Bu platform sayesinde, uygulamaların çalıştırılması için gereken tüm konteynerler bir arada toplanabilir, bu konteynerlerin çalıştırılması için gereken tüm ayarlar ve yapılandırmalar otomatik olarak yapılır. Böylece, uygulamaların dağıtımı ve çalıştırılması için gereken işlemler hızlandırılır ve kolaylaştırılır.

3.5. Mikro Servis Mimarisi

Mikro servis mimarisi, bir uygulamanın tasarımında kullanılan bir yöntemdir. Bu yöntemde, bir uygulama, küçük ve tek bir görev üstlenen birçok parçaya (servisler olarak adlandırılan bu parçalar) ayrılır. Bu servisler, birbirlerine bağımlı olmayan şekilde tasarlandıklarından, ayrı ayrı geliştirilebilir, test edilebilir ve dağıtılabilirler.

Mikro servis mimarisi, bir uygulamanın büyüklüğü ve kompleksliğinin artışına paralel olarak, bu uygulamanın geliştirilmesini ve bakımını kolaylaştırır. Örneğin, bir uygulamanın bir parçasında yapılacak değişiklikler, diğer parçaları etkilemez ve bu sayede uygulamanın bütününde değişiklikler yapılırken güvenlik sağlanır. Ayrıca, mikro servis mimarisi sayesinde, uygulamanın çalıştırılması için gereken servislerin çalıştırılmasını sağlayan ortamlar da ayrı ayrı yapılandırılabilir. Örneğin, bir servis için gereken bellek miktarı farklı bir servis için gereken bellek miktarından daha fazla olabileceği gibi, bir servis için gereken işlemci hızı da farklı bir servis için gereken işlemci hızından daha düşük olabilecektir. Bu sayede, uygulamanın çalıştırılması için gereken kaynaklar daha etkin kullanılır.

Mikro servis mimarisi, ayrıca bir uygulamanın parçalarının farklı programlama dilleriyle yazılmasına da olanak verir. Örneğin, bir servis Python ile yazılırken, diğer bir servis Java ile yazılabilir. Böylece, uygulamanın geliştirilmesinde kullanılan araçlar ve teknolojiler değiştirilebilir ve uygulamanın bütünlüğü bozulmaz. Mikro servis mimarisi, uygulamanın büyüklüğüne ve kompleksliğine göre farklı şekillerde uygulanabilir. Örneğin, bir uygulamanın çok az sayıda ve basit servisi varsa, bu servislerin tümü aynı sunucuda çalıştırılabilir. Ancak, uygulamanın

servisleri sayısı ve görevleri artışıyla birlikte, bu servislerin çalıştırılması için farklı sunucular kullanılabilir. Bu sayede, servisler arasında paylaşılan kaynaklar azaltılır ve uygulamanın çalışma hızı artırılır.

Sonuç olarak, mikro servis mimarisi, bir uygulamanın tasarımında kullanılan bir yöntemdir. Bu yöntem sayesinde, bir uygulama parçalara ayrılır ve bu parçalar birbirlerine bağımlı olmayan şekilde tasarlandıklarından, ayrı ayrı geliştirilebilir, test edilebilir ve dağıtılabilirler.

3.6. MVC Tasarım Deseni

MVC (Model-View-Controller) tasarım deseni, bir uygulamanın tasarımında kullanılan bir yöntemdir. Bu yöntemde, uygulama üç farklı parçaya ayrılır: Model, View ve Controller. Bu parçaların görevleri aşağıdaki gibidir:

Model: Uygulamanın verileri ile ilgili olan parçadır. Örneğin, bir uygulamanın veritabanındaki veriler Model olarak adlandırılır. Model, verilerin depolandığı, saklandığı ve işlendiği parçadır.

View: Uygulamanın kullanıcı arayüzü ile ilgili olan parçadır. Örneğin, bir uygulamanın web sayfası View olarak adlandırılır. View, kullanıcıların verileri görüntüleyebileceği ve uygulamaya veri girebileceği parçadır.

Controller: Uygulamanın işleyişi ile ilgili olan parçadır. Örneğin, bir uygulamanın kullanıcıların veri girdiği zaman verilerin işlenmesi Controller olarak adlandırılır. Controller, Model ve View arasında veri transferi sağlayan parçadır.

MVC tasarım deseni, uygulamanın işleyişini düzenlerken, Model, View ve Controller parçalarının birbirlerinden ayrı olarak tasarlandıklarından, uygulamanın bütününde değişiklikler yapılırken güvenlik sağlanır. Örneğin, uygulamanın veritabanındaki veriler değiştirildiğinde, bu değişiklikler View parçasına etki etmez ve kullanıcıların verileri görüntüleyebileceği arayüzde değişiklik olmaz. Ayrıca, MVC tasarım deseni sayesinde, uygulamanın parçaları birbirlerinden bağımsız olarak geliştirilebilir ve test edilebilir. MVC tasarım deseni, çeşitli programlama dilleriyle (örneğin: Java, Python, PHP gibi) uyumludur ve web tabanlı uygulamaların tasarımında sıklıkla kullanılır. Ayrıca, MVC tasarım deseni sadece web tabanlı uygulamalar için değil, masaüstü ve mobil uygulamalar için de kullanılabilir. Bu yöntem sayesinde, uygulama üç farklı parçaya ayrılır ve bu parçalar birbirlerinden ayrı olarak tasarlandıklarından, uygulamanın geliştirilmesi ve bakımı kolaylaşır. Ancak, MVC tasarım deseninin de dezavantajları vardır ve özellikle büyük ve kompleks uygulamalar için daha yüksek bir öğrenme eğrisine sahip olabilirler.

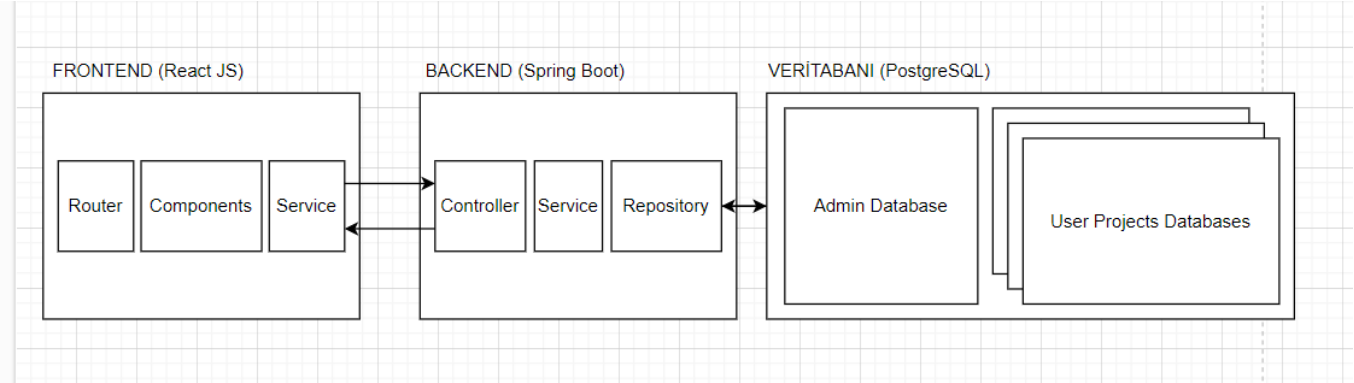
4. Çözüm Önerisi / Yöntem

4.1. Sistem Gereksinim Analizi

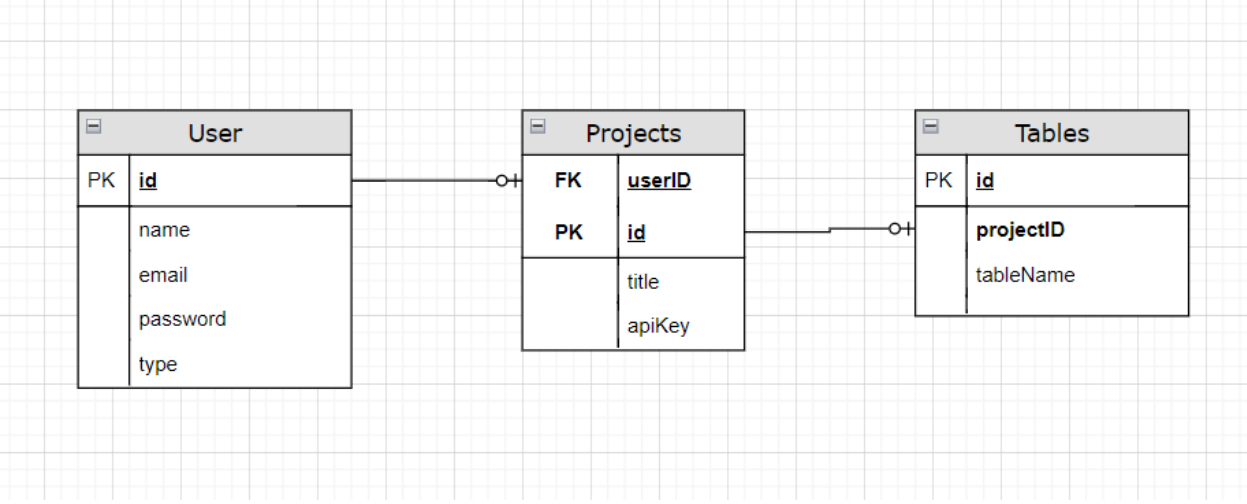
Oluşturacağımız sistem için gereken özellikleri araştırdığımızda ilk olarak Low-Code No-Code bir sistem için gereken ilk özellik olarak kullanıcıların isteklerini görsel olarak çözümlemenin önemini anladık. Kullanıcıların kendilerinin mümkün olan en az kodu yazması için görsel bir arayüz ve anlaması kolay bir kullanım mantığı oluşturmaya çalıştık. Veri tabanlarını oluşturmak ve yönetmek için gereken görsellikleri aşağıda belirttik.

4.2. Sistem Tasarımı

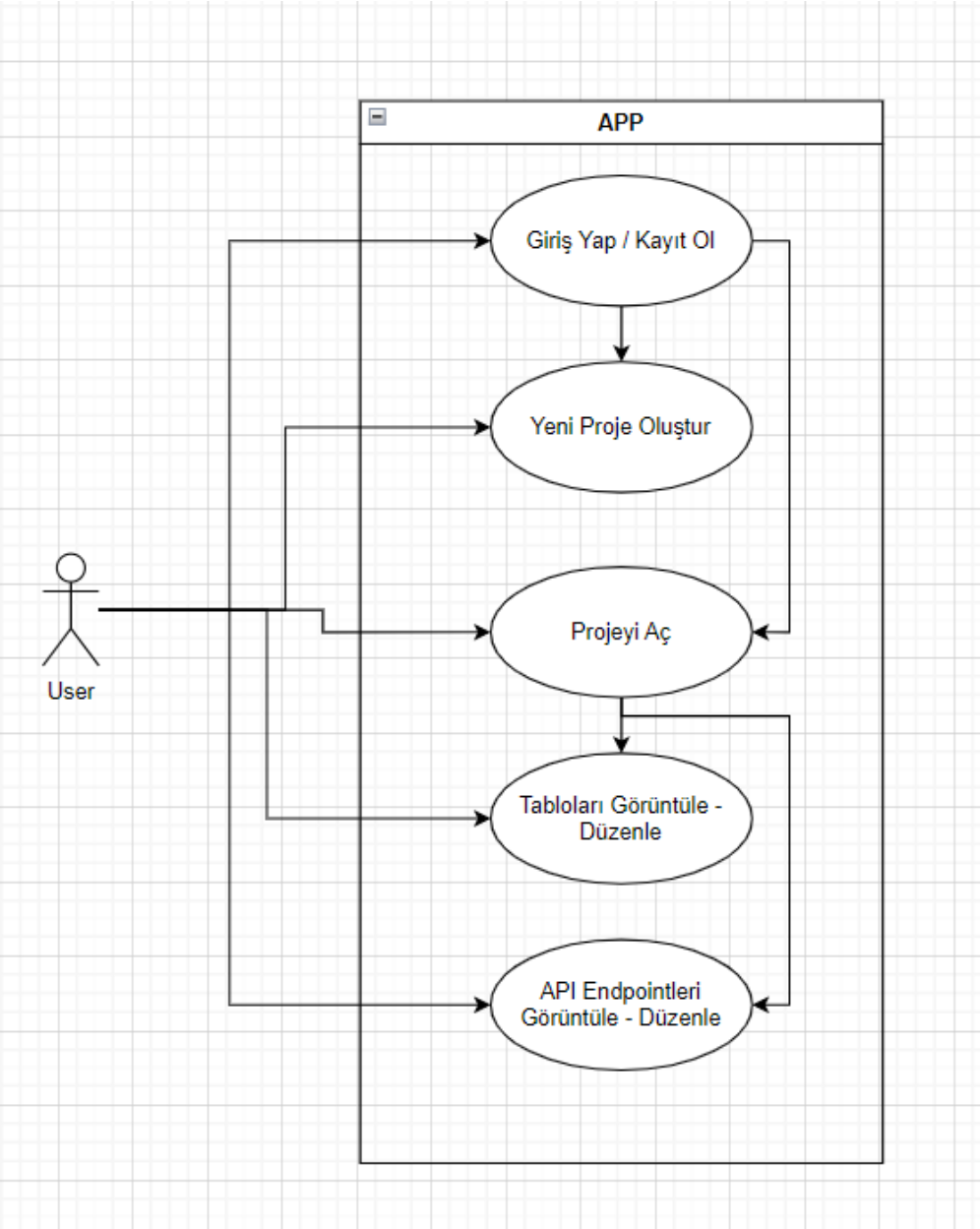
4.2.1. Genel Mimari / Şematik



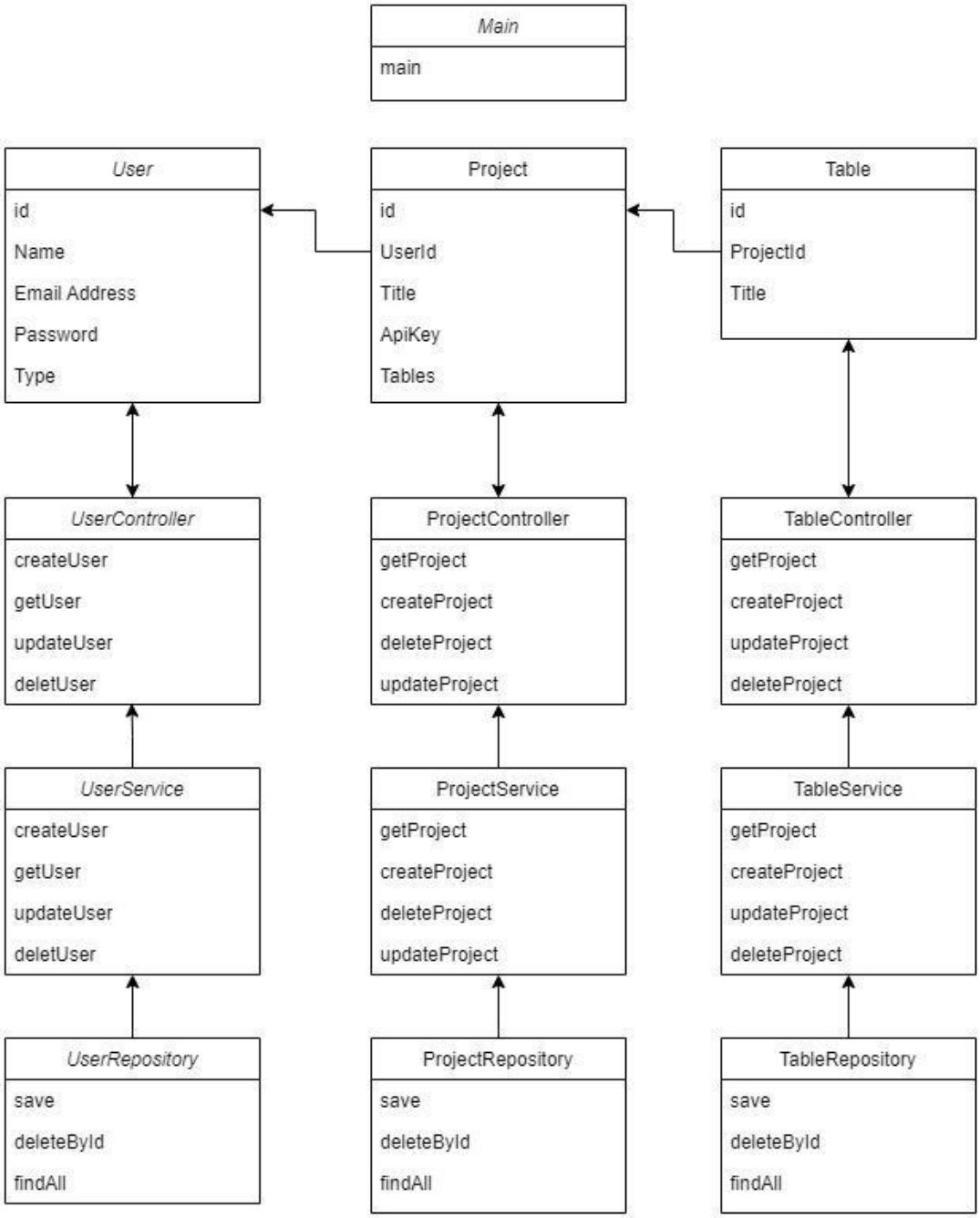
4.2.2. Varlık-Bağıntı (ER) Modeli



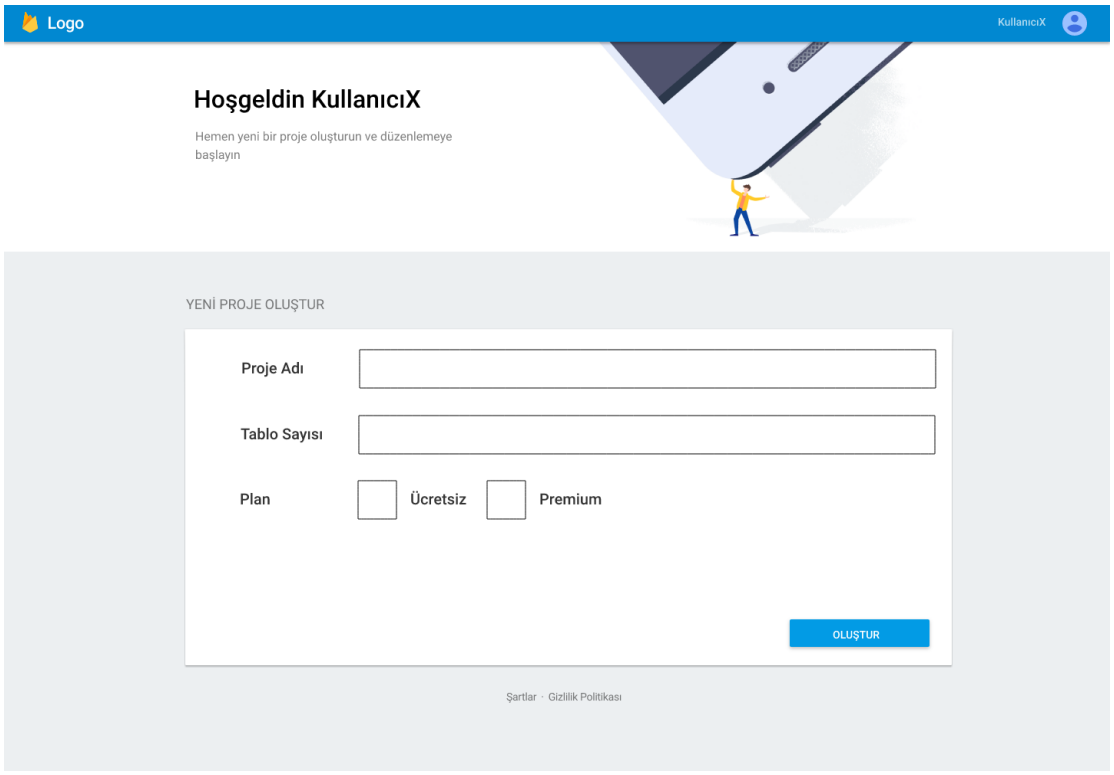
4.2.3. Kullanım Senaryoları (UML)



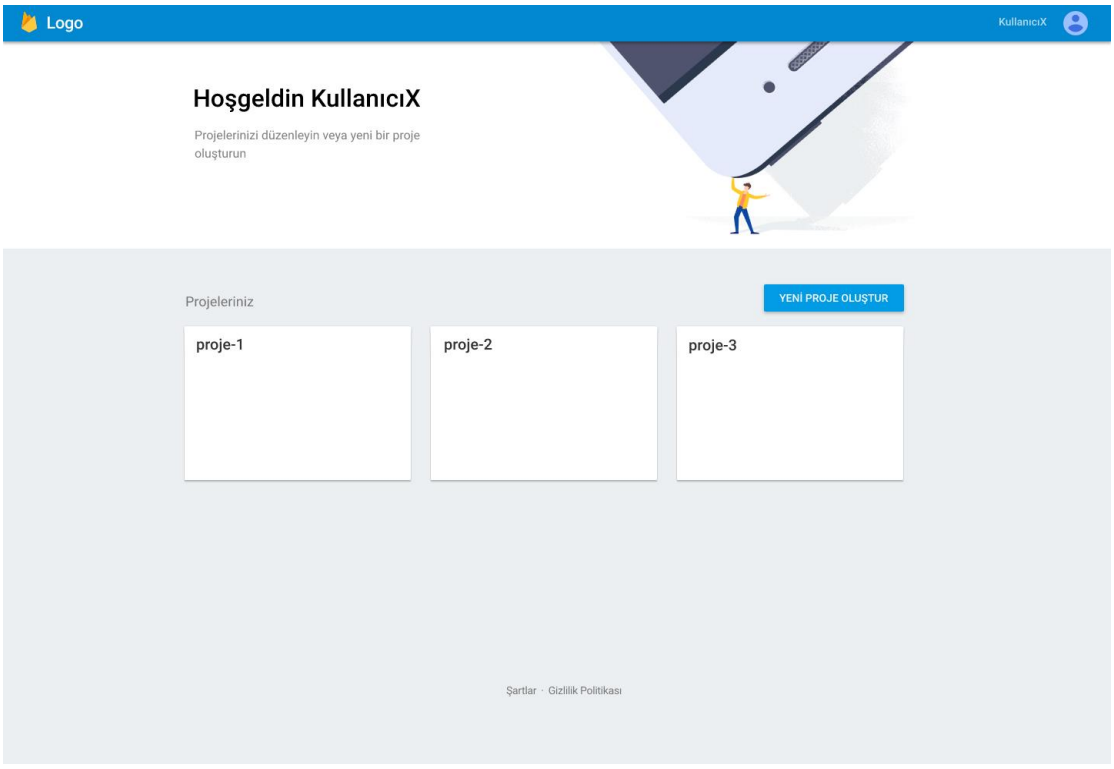
4.2.4. Sınıf Diyagramı (UML)



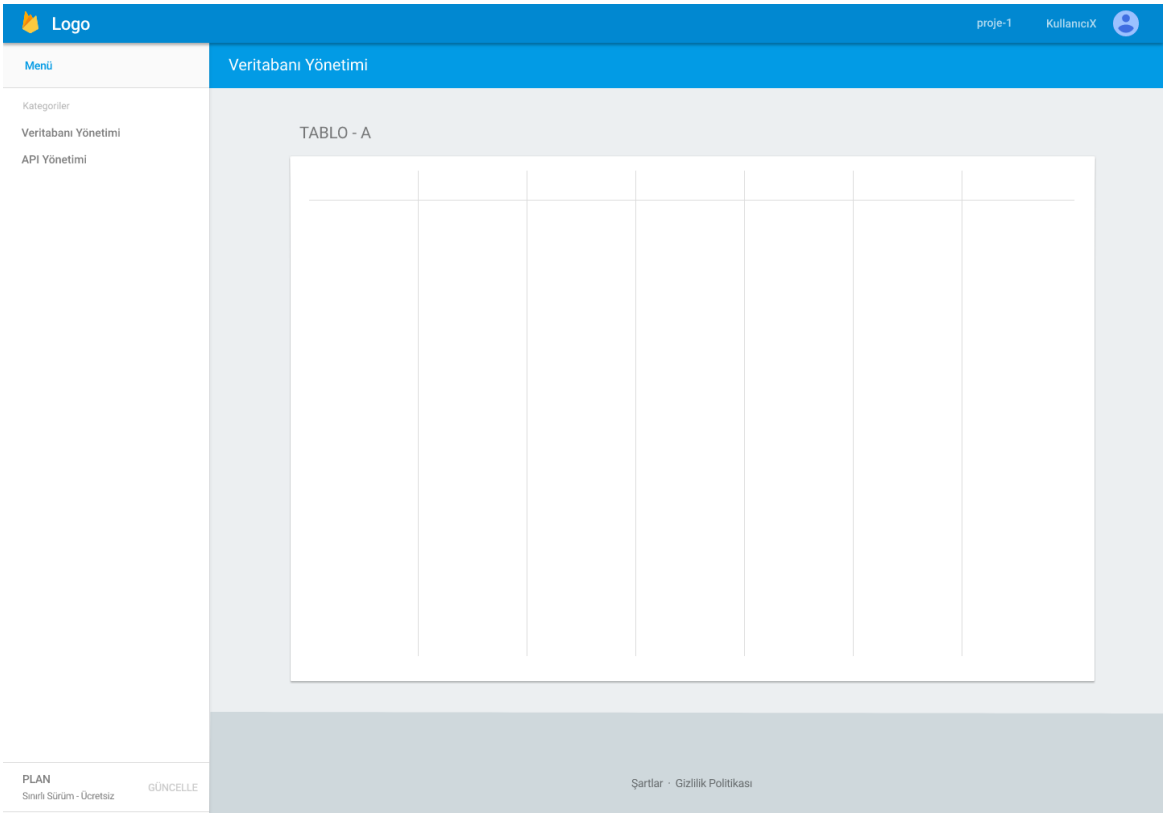
4.2.5. Prototipleme (Wireframing)



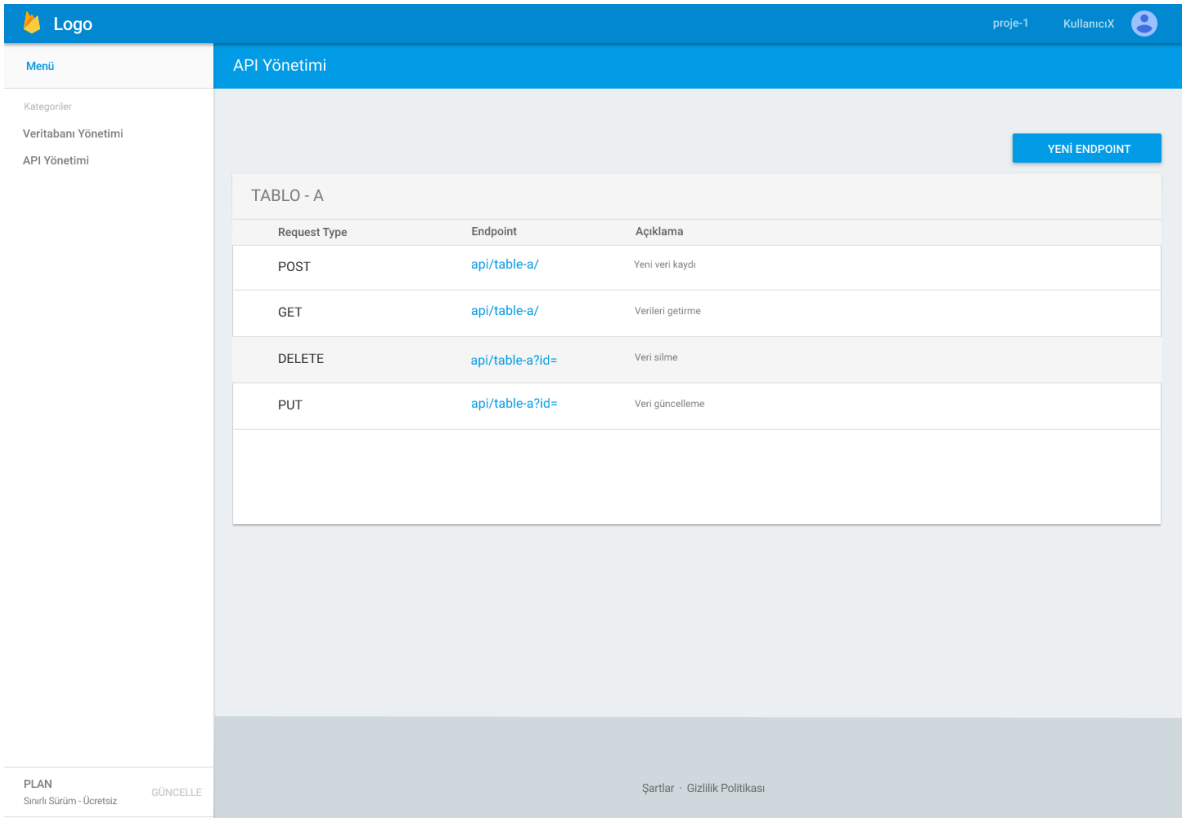
Yeni Proje Oluşturma Sayfası



Ana Sayfa - Tüm Projeler



Proje Tabloları Görüntüleme Sayfası



API Listeleme ve Düzenleme Sayfası

4.3. Kullanılacak Teknolojiler

4.3.1. Backend

Java

Java yüksek seviyeli, sınıf tabanlı, nesneye yönelik programlama dilidir. Java tasarlanırken amacı mümkün olduğu kadar uygulanan ortama olan bağımlılığın azalması olmuştur. Java programcıların bir kere yaz her yerde çalıştır (write once, run anywhere (WORA)) prensibi ile çalışmasını amaçlayan çok işlevli bir dildir. Bunu başarabilmesini sağlayan asıl yöntem ise Java programlama dilinin derlendikten sonra Java destekleyen her platformda tekrar derlenmeye ihtiyaç duymadan çalışabilmesidir. Java uygulamaları genel olarak bytecode'a derlenir ve her Java Virtual Machine (JVM) üzerinde, çalıştığı bilgisayarın mimarisinden bağımsız olarak, çalışabilir. Java'nın sözdizimi (syntax) C ve C++ dillerine benzemektedir, ama bu dillere göre daha az düşük seviye olanaklar içerir. Java diğer dillerin çoğunda olmayan bir özellik olarak çalışma zamanında dinamik kabiliyet gösterebilir (çalışma zamanında kod modifikasyonu gibi).

Java Programlama Dili Hangi Alanlarda Kullanılır?

1. Oyun Geliştirme

Pek çok popüler video oyunu mobil platformlar da dahil olmak üzere Java üzerinde geliştirilmiştir. Günümüzdeki makine öğrenmesini kullanan veya VR üzerinde çalışan modern oyunların bile Java ile geliştirilmeleri mümkündür.

2. Bulut Bilişim

Java bir kere yaz, her yerde çalıştır prensibi ile dağıtık bulut tabanlı uygulamalar için harika bir uyuma sahiptir. Bulut sistemleri geniş ölçekli bir destek için sıklıkla Java'yı tercih etmekteler.

3. Büyük Veri

Java karmaşık veri setleri ve devasa boyutlara ulaşan gerçek zamanlı veriler ile çalışan veri işleme motorları tarafından kullanılmaktadır.

4. Yapay Zekâ

Java hızı ve stabil olması sayesinde yapay zekâ uygulamaları için mükemmel bir dildir.

5. Nesnelerin İnterneti

Java ile kendi başına internete bağlanabilen donanımlar ve sensörler programlanabilmektedir.

Spring Framework

Spring Java ve .NET için geliştirilmiş açık kaynak kodlarına sahip bir framework'tür. Spring Framework'ü ilk olarak Rod Johnson tarafından yazıldı ve 2003 Haziran ayında yayınlandı. Framework genel olarak farklı teknik problemler için çözüm sunan bir altyapıdır

denilebilir. Spring Framework'ün çekirdek özellikleri herhangi bir Java uygulaması tarafından kullanılabilir ama uzantılar sayesinde Java EE'nin üzerinden web uygulamaları geliştirmek için de kullanılmaktadır. Spring Framework'ü Java tabanlı uygulamalar için kapsamlı bir programlama ve biçimlendirme modeli sunmaktadır. Java ile yazılan programlar epey karmaşık olabilirler. Spring hafif (lightweight), güvenli ve esnek bir framework'tür. Bu avantajları sayesinde Spring ile kod yazarken verimlilik daha fazladır ve genel olarak geliştirme sürecinde harcanan zaman daha azdır. Spring sayesinde var olan iş gücü sıkıcı düzenleme işlemleri yerine doğrudan iş mantığına harcanabilir. Spring Framework'ü kodun altyapısını oluşturacağı için geliştiriciler uygulamanın kendisine odaklanabilirler.

Spring Kullanmanın Avantajları

· Spring enterprise uygulamaların POJO'ları (plain old java object) kullanarak geliştirilebilmesinin önünü açar.

· Spring modüler bir tasarıma sahiptir. Kullanılan paketlerin ve sınıfların sayısı bazen oldukça fazla olsa bile modülerliği sayesinde sadece ihtiyacımız olan sınıflarla ilgilenmemiz gerekir, geri kalanı görmezden gelebiliriz.

· Spring sayesinde zaten var olan ORM frameworkleri ve JDK zamanlayıcıları gibi teknolojileri kullanabiliriz ve tekrar yazmamız gerekmez.

· Spring ile yazılmış olan kodu test etmek oldukça kolaydır çünkü ortama bağlı olan kod bu framework içerisine taşınmıştır.

· Spring oldukça istikrarlı bir işlem yönetim arayüzü sunar bu sayede hem küçük ölçekte hem de global ölçekte işlemlere olanak sağlar.

· Spring aslında framework'ler için bir framework'tür diyebiliriz çünkü Struts, Hibernate, Tapestry, EJB, JSF gibi birden çok framework için destek sunan bir framework'tür.

Spring Boot

Spring Boot yukarıda bahsedilen ve oldukça popüler olan Spring Framework ile mikroservisler ve web uygulamalarını daha az zamanda ve daha kolay şekilde geliştirmek için kullanılan bir araçtır. Spring Boot üç ana özelliği ile öne çıkar:

1. Otomatik Yapılandırma (Autoconfiguration)

2. Biçimlendirmeye karşı inatçı bir yaklaşım

3. Bağımsız uygulamalar geliştirebilme yeteneği

Bu özellikleri ile Spring Boot, Spring tabanlı uygulamaları minimal biçimlendirme ve kurulum ile oluşturmak için harika bir araç olur.

Spring Boot Spring Framework'üne Neler Katar?

Spring Framework oldukça kapsamlı ve yetenekli bir framework olsa bile büyük dezavantajları vardır. Spring ile uygulama geliştirmek için geliştiricilerin bu framework'e olan hakimiyetleri yüksek olmalıdır ve geliştirme için harcanan zaman oldukça fazla olabilir. Spring

Boot yukarıdaki özellikleri sayesinde harcanan eforu ve zamanı büyük ölçüde azaltır. Bu özellikleri açmak gerekirse:

1. Otomatik Yapılandırma (Autoconfiguration)

Otomatik yapılandırma demek uygulamaları önceden belirlenen ve manuel olarak biçimlendirmeye gerek kalmayan bağımlılıkları ile uygulamaların başlatılabilir olması demektir.

2. İnatçı Yaklaşım

Spring Boot geliştiriciye bırakmak yerine kendi yaklaşımını kullanarak geliştirilen uygulamalar için hangi paketleri yükleyeceğini ve hangi varsayılan değerleri kullanacağını kendi başına seçer.

3. Bağımsız Uygulama Geliştirebilme

Spring Boot geliştiricilerin 'hemen çalışan' uygulamalar geliştirebilmelerine olanak sağlar. Özellikle açmak gerekirse kendi başına çalışabilen başlatma sırasında dışarıdan bir server vb. yardımlara gerek duymayan uygulamalar oluşturulabilir.

4.3.2. Frontend

HTML

Açılımı 'HyperText Markup Language' (Hiper Metin İşaretleme Dili) olan HTML web sayfalarında oluşturacağımız içeriğin yapısını belirlemekte kullandığımız işaretleme dilidir. İşaretleme dili, metinlerin nasıl yapılandırılacağı veya biçimlendirileceği hakkında talimatlar içeren bir dil olarak tanımlanır. HTML içerdiği bir dizi öğe ile oluşturmak istediğimiz web sayfasının içeriğinde bölümleri düzenlememize olanak sağlar. HTML oluşturulan dokümanların web tarayıcılarında görüntülenmesini sağlar. CSS ve JavaScript gibi teknolojilerle beraber kullanılarak dinamik web sayfaları oluşturulabilir. Ayrıca HTML'in bir programlama dili olmadığına dikkat etmek gerekir. Programlama dili olmamasının sebeplerinden birisi HTML kullanarak kendi başına çalışan programlar oluşturamıyor olmamızdır. HTML kullandığımızda bir web tarayıcısı ile oluşturulan dosyanın görüntülenmesi gerekmektedir. HTML artık resmi olarak bir web standardı olarak kabul görmektedir. HTML artık 'World Wide Web Consortium' (W3C) tarafından geliştirilmekte ve düzenli olarak güncellenmektedir. HTML'in birçok kullanım senaryosu vardır:

- Web Geliştirme:
  - Geliştiriciler HTML kullanarak bir web sayfasının tarayıcı tarafından nasıl görüneceğini belirlerler.
- İnternet Yönlendirmesi:
  - Geliştiricilerin HTML belgelerine gömdükleri bağlantılar sayesinde kullanıcılar ilgili sayfalar arasında kolayca dolaşabilirler.

HTML öğeleri aşağıdaki gibi oluşturulabilir:

- Açılış Etiketi:

- Burası kullanılacak olan etiketin adını içerir ve büyüktür küçüktür işaretleri ile sarılı olur.

· Kapanış Etiketi:

- Bu etiket '/' işaretinin etiketin başında kullanılması haricinde açılış etiketinin aynısıdır.

· İçerik:

- Açılış ve kapanış etiketleri arasına HTML öğesinin içeriğini yazarız.

CSS

CSS, HTML (Hypertext Markup Language) dilinde yazılmış bir belgenin görünümünü ve biçimlendirilmesini tanımlayan bir stil sayfası dilidir. Web sayfalarının renkleri, düzeni ve yazı tipini denetlemek için kullanılır. CSS, bir web sayfasının tasarımını düzenlerken farklı özellikler kullanılabilir. Örneğin, bir web sayfasındaki yazıların rengi, tipi, boyutu, hizalaması, satır aralığı gibi özellikleri CSS ile tanımlanabilir. Ayrıca, bir web sayfasındaki resimlerin pozisyonu, boyutu ve çerçevesi gibi özellikleri de CSS ile tanımlanabilir.

CSS, web sayfasındaki HTML elementlerine (başlıklar, paragraflar ve bağlantılar gibi) stil uygulamanıza olanak verir. Bir website'nin birden fazla sayfasında tutarlı stil oluşturabilir veya tek bir sayfanın görünümünü değiştirebilirsiniz. CSS kuralları seçiciler ve bildirimlerden oluşur. Bir seçici, bir dizi bildirime uygulanacak HTML elementlerini tanımlamak için kullanılır. Bildirimler, seçilen elementlere uygulanacak stili tanımlayan özellikler ve değerlerden oluşur. Örneğin, bir sayfadaki tüm başlıkların yazı tipini diğer metnin üstünde büyük olarak ayarlamak için aşağıdaki gibi bir CSS kuralı kullanabilirsiniz:

```
h1, h2, h3 {

    font-size: larger;

}
```

CSS stilini bir web sayfasına birçok farklı yolla uygulayabilirsiniz, bunlar arasında inline stil (HTML elementinde style özniteliği kullanarak), internal stil (HTML belgenin başında <style> elementi kullanarak) ve external stil (HTML belgesine bağlı ayrı bir CSS dosyası kullanarak) yer alır. CSS, ayrıca bir web sayfasının farklı cihazlarda farklı görünümler oluşturulmasını da sağlar. Örneğin, bir web sayfasının bilgisayarda görünümü farklı olabileceği gibi, aynı web sayfasının mobil cihazlarda görünümü de farklı olabilecektir. Bu sayede, web sayfalarının cihazlara göre optimize edilmesi sağlanır ve web sayfalarının kullanımı daha kolay hale gelir.

Bootstrap

Bootstrap, web sayfaları ve uygulamaları geliştirmek için kullanılan bir araçlar kümesi ve önyüz çatısıdır. HTML ve CSS tabanlı tasarım şablonları içerir ve bu şablonlar sayesinde formlar, navigasyon çubukları, düğmeler gibi arayüz bileşenleri oluşturulabilir. Bootstrap, HTML, CSS, Less, Sass ve JavaScript kullanılarak geliştirilmiştir ve tamamen etkileşimli ve duyarlı web uygulamaları oluşturmak için kullanılabilir. Web uygulamalarının ön uç iskeleti olarak düşünülebilir, çünkü HTML kodlarıyla yapıyı oluşturur, stil şablonlarıyla görünümü belirler ve

JavaScript kodlarıyla dinamik öğeleri ekler. Bootstrap, Twitter mühendisleri Mark Otto ve Jacob Thornton tarafından geliştirilmiştir ve web uygulamalarını hızlı, kullanıcı dostu ve duyarlı hale getirmeyi amaçlar. Mobil uyumluluk için tasarlanmış duyarlı ve mobil öncelikli web siteleri oluşturmak için yaygın olarak kullanılan bir CSS iskeletidir. Bootstrap, farklı cihazlarda uyumlu olacak şekilde tasarlanmıştır ve web sitesinin otomatik olarak farklı ekran boyutlarına uyum sağlaması sağlar. Ayrıca, web geliştirme sürecini hızlandırırken kaliteden ödün vermez. HTML ve CSS bilgisine sahip olan herkes Bootstrap kullanarak web siteleri, uygulamalar ve temalar geliştirebilir. Bootstrap ile geliştirilen web siteleri modern tarayıcılarda tutarlı bir şekilde görüntülenir ve Bootstrap'in açık kaynaklı ve ücretsiz olması da büyük bir avantajdır. Geliştiriciler, Bootstrap üzerinde oluşturdukları kaynakları ücretsiz veya ücretli olarak diğerlerinin kullanımına sunabilir ve bu kaynaklar web siteleri ve uygulamaların geliştirilmesinde kullanılabilir.

Günümüzde insanlar çeşitli donanım özelliklerine, işletim sistemlerine, ekran boyutlarına ve etkileşim şekillerine sahip birçok farklı cihaz kullanıyor. Bir web sitesinin dizüstü bilgisayarlardan akıllı telefonlara kadar tüm cihazlarla uyumlu olması, günümüzde bir gerekliliğe dönüştü. Bootstrap ile oluşturulan web sitelerinin arayüzü hem bugün kullanılan hem de yarın çıkabilecek tüm ekran boyutlarına kendiliğinden uyarlanır. Bu uyumluluk, masaüstü deneyiminde derli toplu görünen bir web sayfasının, mobil bir cihazda da sorunsuz görüntülenebilmesi ve etkileşime geçilebilmesi anlamına gelir. Üstelik, Bootstrap ile web geliştiriciliğinin bunun dışında da birçok avantajı var.

Bootstrap, web geliştirme projelerini hızlandırarak kalite standartlarından ödün vermeden ilerlemeyi sağlayan en büyük avantajlardan birine sahiptir. Bootstrap olmadan duyarlı bir web sitesi veya uygulama geliştirmek uzun sürebilirken, bu araç seti süreyi büyük ölçüde kısaltır. Başlangıç düzeyinde HTML ve CSS bilgisine sahip olan herkes, Bootstrap kullanarak web siteleri, uygulamalar ve temalar geliştirebilir. Bootstrap ile geliştirilen duyarlı web siteleri otomatik olarak tüm ekran boyutlarına uyum sağlar. Firefox, Chrome, Edge, Opera ve Safari gibi modern web tarayıcılarında, Bootstrap ile tasarlanmış bir web sayfası aynı şekilde görüntülenir. Bootstrap'in en önemli özelliği açık kaynaklı ve tamamen ücretsiz olmasıdır. Birçok geliştirici, bu platformda oluşturdukları kaynakları ücretli veya ücretsiz olarak diğerlerinin kullanımına sunar. Bu kaynaklar, web siteleri ve uygulamalar geliştirmek için kullanılabilir.

Javascript

JavaScript, HTML ve CSS ile birlikte webin temel teknolojilerinden biri olan bir programlama dilidir. Web sitelerinin büyük çoğunluğu, kullanıcı tarafında web sayfasının hareketliliğini sağlamak için JavaScript kullanır ve genellikle üçüncü taraf kütüphanelerden oluşan kodlar içerir. Tüm yaygın web tarayıcılarında, JavaScript kodunu kullanıcıların cihazlarında çalıştırmak için özel bir JavaScript motoru bulunur. Diğer programlama dilleri gibi, JavaScript de işletim sistemi tarafından çalıştırılabilen makine koduna çevrilir. JavaScript genellikle bir betik dili veya yorumlanan bir dil olarak sınıflandırılır. JavaScript kodu yorumlanır, yani bir JavaScript motoru tarafından doğrudan temel makine diline çevrilir. Diğer programlama dillerinde olduğu gibi, tüm betik dilleri programlama dilleridir, ancak tüm programlama dilleri betik dili olmayabilir.

JavaScript, ECMAScript standardına uyumlu, genellikle eşzamanlı olarak yorumlanan ve üst düzey bir programlama dilidir. Dinamik yazma özelliğine sahip olan JavaScript, prototip tabanlı nesne yönelimine ve birinci sınıf işlevlere sahiptir. Çoklu paradigmayı destekleyen bir dil olarak, olay odaklı, işlevsel ve zorunlu programlama stillerini benimser. Metin, tarihler, düzenli ifadeler, standart veri yapıları ve DOM (Belge Nesne Modeli) gibi API'ler aracılığıyla çalışma

yeteneği sunar. ECMAScript standardı, ağ oluşturma, depolama veya grafik gibi I/O (giriş/çıkış) işlevlerini doğrudan içermez. Ancak, JavaScript API'leri, web tarayıcıları veya diğer çalıştırma ortamları tarafından I/O işlemleri için sağlanır.

JavaScript motorları başlangıçta yalnızca web tarayıcılarında kullanılırken, günümüzde bazı sunucu sistemleri ve çeşitli uygulamalar için de temel bir bileşen haline gelmiştir. Bu kullanım için en popüler çalışma zamanı sistemi Node.js'tir. Java ve JavaScript, adları, sözdizimleri ve bazı standart kitaplıkları açısından benzerlik gösterse de, iki dil arasında büyük farklılıklar vardır ve tasarım açısından da ayrıdırlar. JavaScript'in sözdizimi öğrenmesi ve kodlaması kolaydır. Geliştiriciler, neredeyse her web sitesinde ve mobil uygulamada istemci tarafı komut dosyaları oluşturmak için JavaScript'i kullanır.

Diğer programlama dillerinden farklı olarak, JavaScript'i herhangi bir web sayfasına ekleyebilir ve diğer birçok web geliştirme çerçevesi ve dil ile birlikte kullanabiliriz. JavaScript kodunu yazdıktan sonra herhangi bir makinede çalıştırabiliriz, bu da JavaScript'i uygulama geliştirme platformu bağımsız hale getirir. JavaScript'i sunucu yükünü azaltmak ve ağ trafiğini azaltmak için kullanabiliriz, çünkü mantıksal işlemleri çalıştırabilir ve sunucunun çoğu işini istemci tarafında gerçekleştirebilir. Örneğin, bir kayıt formu doldurma sürecini düşünelim. JavaScript hızlı bir şekilde cep telefonu alanına girilen sayının 10 basamaklı olup olmadığını kontrol edebilir. Bu istekler sunucuya gönderilirse, her hata için sayfa yeniden yüklenir ve kayıt işlemi yavaş ve zahmetli hale gelir. JavaScript aynı anda birkaç farklı talimat setini paralel olarak çalıştırabilir.

React

React (ReactJS veya React.js olarak da bilinir) kullanıcı arayüzü oluşturmaya yarayan en popüler açık kaynak kodlu Javascript kütüphanelerinden birisidir. Web siteleri işlemek için kullanıcı çıktısına harika yanıt sunan bir yöntemi kullanır. Facebook önderliğinde bir geliştirici grubu tarafından geliştirilmekte olan React, Model-View-Controller prensibine uygun olarak oluşturulmuştur. React ile single-page olarak adlandırılan sayfalar geliştirilebileceği gibi React-Native ile mobil uygulamalar da geliştirilebilir. Şu anda, çok satan listesinin en üstünde yer alan Angular ve Bootstrap gibi ana rakiplerinin önündedir.

ReactJS geliştirilmeden evvel, geliştiriciler kullanıcı arayüzleri geliştirirken -Vanilla JavaScript- veya -jQuery- gibi diğer JavaScript kütüphanelerini kullanıyordu. Bu kütüphaneler ise daha az kullanıcı arayüzü odaklıydı. Dolayısıyla birçok hata ve bug ile karşılaşılıyor, bu da sürecin zorlaşmasına ve vakit kaybına sebep oluyordu.

React, açık kaynaklı bir JavaScript GUI kütüphanesidir ve temel odak noktası UI görevlerini verimli bir şekilde tamamlamaktır. MVC kalıbındaki (Model-View-Controller - Model-Görünüm-Kontrolcü) "V" kısmı olarak sınıflandırılabilir. Herhangi bir web sitesinin temelinde HTML belgeleri bulunur. Web tarayıcısı bu belgeleri okur ve bilgisayarınızın veya diğer elektronik cihazlarınızın ekranına web sayfaları olarak yansıtır. Bu süreçte tarayıcılar, sayfaların düzenini tanımlayan bir yapı olan DOM (Belge Nesne Modeli) oluşturur. Geliştiriciler, bu yapıyı JavaScript gibi programlama dilleriyle değiştirerek dinamik içerikler ekleyebilir. React bileşeni, HTML ile JavaScript'i birleştirmeyi sağlayan ve daha kolay yazılmasını sağlayan bir sözdizimi uzantısı olan JSX'i kullanır. JSX, JavaScript ile HTML'in mükemmel bir karışımıdır ve web sitesi yapısını yazma sürecini açıklar. Ayrıca, kullanılan bu uzantı, birden çok fonksiyonun işlenmesini kolaylaştırır. JSX, en popüler sözdizimi uzantısı olmasa da özel bileşenlerin veya büyük ölçekli uygulamaların geliştirilmesinde oldukça verimli olduğunu kanıtlamıştır.

Eğer ReactJS kullanılmıyorsa, bir web sitesi DOM'u güncellemek için doğrudan HTML kullanır. Bu, kullanıcının arayüzle etkileşimi olmadan ekran üzerinde değişikliklerin gerçekleştiği bir işlemdir. Bu süreç statik web sitelerinde sorunsuz bir şekilde çalışırken, kullanıcı yoğunluğunun yüksek olduğu dinamik web sitelerinde kullanıcı her yenileme komutu verdiğinde tüm DOM'un güncellenmesi gerekmektedir. Bu durum ciddi performans sorunlarına yol açabilir.

React ise Sanal DOM (Virtual Document Object Model) mimarisini kullanarak, render işlemini hızlı bir şekilde gerçekleştiren bir kütüphanedir. Bu nedenle, değişikliklerin sıkça olduğu projelerde kullanılmaktadır. Özellikle kullanıcı arayüzünün yoğun olarak değiştiği projeler için uygundur. React, çeşitli arama motorları tarafından taranabilir bir kullanıcı arayüzü oluşturmak için de kullanılabilir. Bu sayede web siteleri, SEO (Arama Motoru Optimizasyonu) açısından daha iyi bir performans gösterebilir.

4.3.3. Servisler

PostgreSQL

PostgreSQL veya Postgres, açık kaynak kodlu bir ilişkisel veritabanı yönetim sistemidir. SQL desteği sayesinde güçlü bir şekilde veritabanı işlemlerini gerçekleştirebilir. PostgreSQL, ücretsiz olarak kullanılabilen ve birçok avantaja sahip bir veritabanı sistemidir.

PostgreSQL, SQL dilinin güvenlik, depolama ve ölçeklendirme özelliklerini etkin bir şekilde kullanır. Bu nedenle, verilerin güvenli ve hatasız bir şekilde depolanmasını sağlar. PostgreSQL, geliştiriciler ve proje yöneticileri için verileri yönetme ve depolama konusunda büyük bir yardımcıdır.

Açık kaynak olması, PostgreSQL'i esnek ve uyarlanabilir kılar. Kullanıcılar, ihtiyaçlarına göre PostgreSQL'i özelleştirebilir ve projelerine uygun şekilde kullanabilir. PostgreSQL'in güçlü özellikleri ve açık kaynak avantajı, veritabanı yönetimi konusunda birçok alanda tercih edilen bir seçenek haline gelmesini sağlar.

PostgreSQL, yüksek performans ve kararlı bir mimari sunarak SQL standartlarına uyumlu bir şekilde çalışmayı hedefler. SQL'nin birçok önemli özelliğini desteklemesine rağmen bazı syntax fonksiyonlarında küçük farklılıklar bulunabilir. PostgreSQL, veritabanı mimarisine ihtiyaç duyulan birçok alanda kullanılabilir.

PostgreSQL, çeşitli işletim sistemleri tarafından desteklenir. Bu nedenle PostgreSQL'i Windows, MacOS, Solaris, SuSE ve OpenSuSE, Ubuntu Linux ve türevlerinde, Debian GNU/Linux ve türevlerinde, Red Hat Linux ailesinde (CentOS/Fedora/Scientific/Oracle ve türevlerinde) ve diğer Linux dağıtımlarında, OpenBSD ve FreeBSD üzerinde kullanabilirsiniz.

Ayrıca, PostgreSQL birçok programlama dili tarafından da desteklenmektedir. C (libpq), Gömülü C (ecpg), Python (PyGreSQL), Java (JDBC) ve TCL (libpgtcl) gibi programlama dilleriyle birlikte kullanılabilir.

PostgreSQL'in geniş işletim sistemi ve programlama dilinde desteklenmesi, kullanıcıların çeşitli platformlarda ve dillerde rahatlıkla PostgreSQL'i kullanabilmesini sağlar. PostgreSQL, rakipleri karşısında en güçlü kılan özelliklere sahiptir. Transaction, subselect, trigger, view, foreign key referential integrity, user-defined types, rules, inheritance ve sophisticated locking

gibi özellikler PostgreSQL'in diğer veritabanlarından ayıran özelliklerdir. Ayrıca, multi-version uyumluluk özelliği sayesinde lock çakışmalarını azaltır.

Performans açısından PostgreSQL, diğer ticari veya açık kaynak veritabanlarıyla kıyaslandığında başarılı sonuçlar elde eder. Diğer veritabanı sistemlerine göre bazı durumlarda daha hızlı, bazı durumlarda ise daha yavaş olabilir. Örneğin, INSERT/UPDATE işlemlerinde transaction bazlı çalışması nedeniyle diğer veritabanlarına göre daha yavaş olabilir. Ancak, yukarıda belirtilen özelliklerin birçoğu MySQL ve benzeri veritabanlarında bulunmayabilir.

PostgreSQL, güvenlik açısından da başarılı bir veritabanıdır. Hızlı olmasından ziyade güvenilirlik üzerine odaklanmaktadır. Stabil çalışan sürümler yayınlanmadan önce uzun süreli beta testleri gerçekleştirilir ve hatasız bir şekilde kodlanmaya çalışılır. Bu sayede PostgreSQL kullanıcılarına güvenilir bir hizmet sunulur.

Sonuç olarak, PostgreSQL bazı yönlerden MySQL ve benzeri veritabanlarından geride kalabilir, ancak sunduğu özellikler, güvenilirlik ve esneklik açısından önemli avantajlara sahiptir.

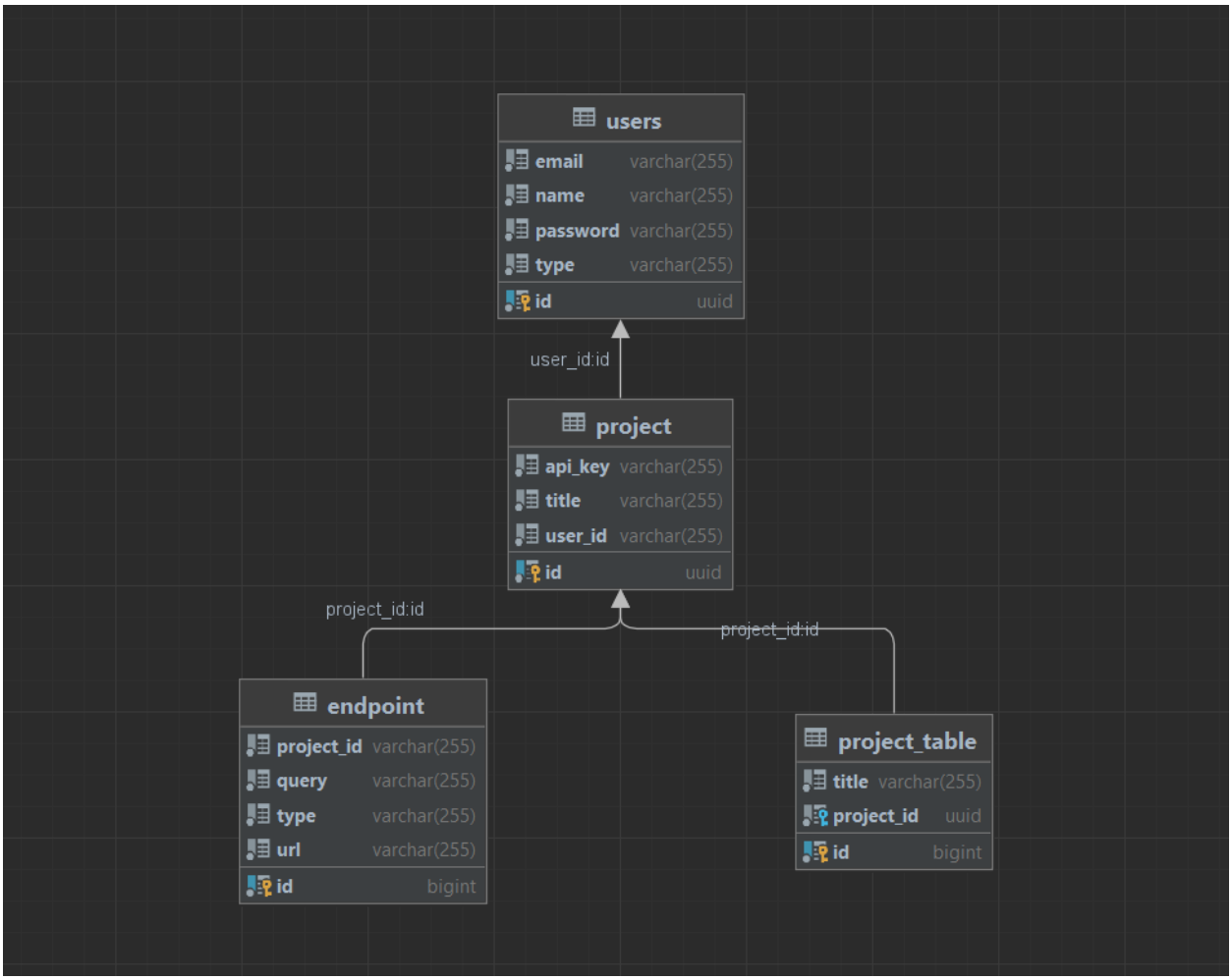
5. Sonuç / Tartışma

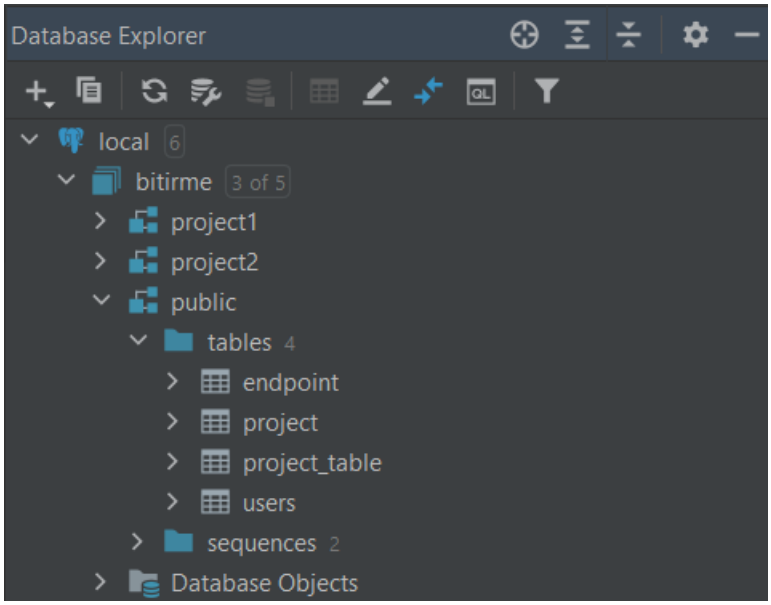
Yaptığımız araştırmalar sonucu yukarıda tanımını verdiğimiz problemi çözmek için bir kod yapısı oluşturduk. Bu yapıyı oluşturmak için kullanacağımız teknolojileri sıraladık ve oluşturmak istediğimiz kod için diyagramlarla mantığı anlattık.

İlk olarak benzer sistemlerin özelliklerini araştırarak başladık. Oluşturulan sistemlerin hangi tür kullanıcılar için çözüm ürettiğini bulduktan sonra bizim yönelmek istediğimiz alanı seçtik. Problem için kendi getirdiğimiz mantığı anlattıktan sonra ise diyagramlar ile oluşturmak istediğimiz algoritmaları özetledik.

Uygulama

Projeyi hedeflerimiz ölçüsünde gerçekleştirmeye başladık. Backend uygulaması için belirlediğimiz gibi Java ile bir Spring Boot projesini Intellij IDEA üzerinde oluşturduk. Gerekli bağımlılıkları ekledik. Veritabanı için PostgreSQL kurulumu yaptık. Bağlantı bilgilerini projemize ekledik ve test bağlantısı yaptık. Tasarladığımız tabloları ve ilişkileri oluşturduk. Spring Boot uygulamasında gerekli tablolara karşılık gelen modelleri yazdık. Programın çalışma kurgusuna uygun olarak gereken servis sınıflarını ve endpointleri oluşturduk.





Veritabanına test için örnek datalar ekledik. Postman üzerinden oluşturduğumuz servislere test istekleri gönderdik ve veritabanı üzerinden işleyişi takip ettik. Servislerde kullanıcı oluşturma ve okuma, yeni proje oluşturma ve listeleme, endpoint ve tablolar oluşturma ve listeleme için metotlar oluşturduk. Kullanıcının yeni bir proje oluşturması durumunda veritabanında public dışında bir şema oluşturulur. Bu şemanın bilgileri project tablosunda saklanır.

```java
@CrossOrigin("http://localhost:3000")
@RestController
@RequestMapping("/user")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) { this.userService = userService; }

    @GetMapping
    public ResponseEntity<UserDto> getUser(String email, String password) {
        return ResponseEntity.ok(userService.getUser(email, password));
    }

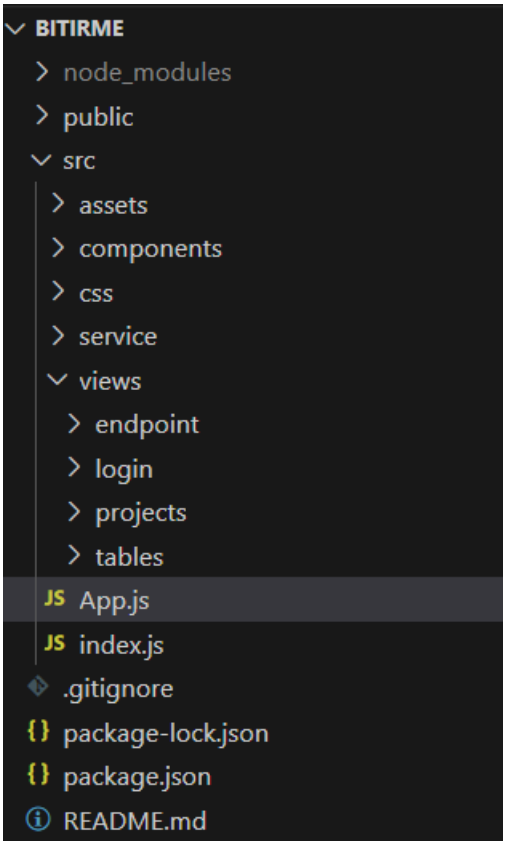
    @PostMapping
    public ResponseEntity<Object> createUser(@Valid @RequestBody CreateUserRequest user) {
        return ResponseEntity.ok(userService.createUser(user));
    }

}
```

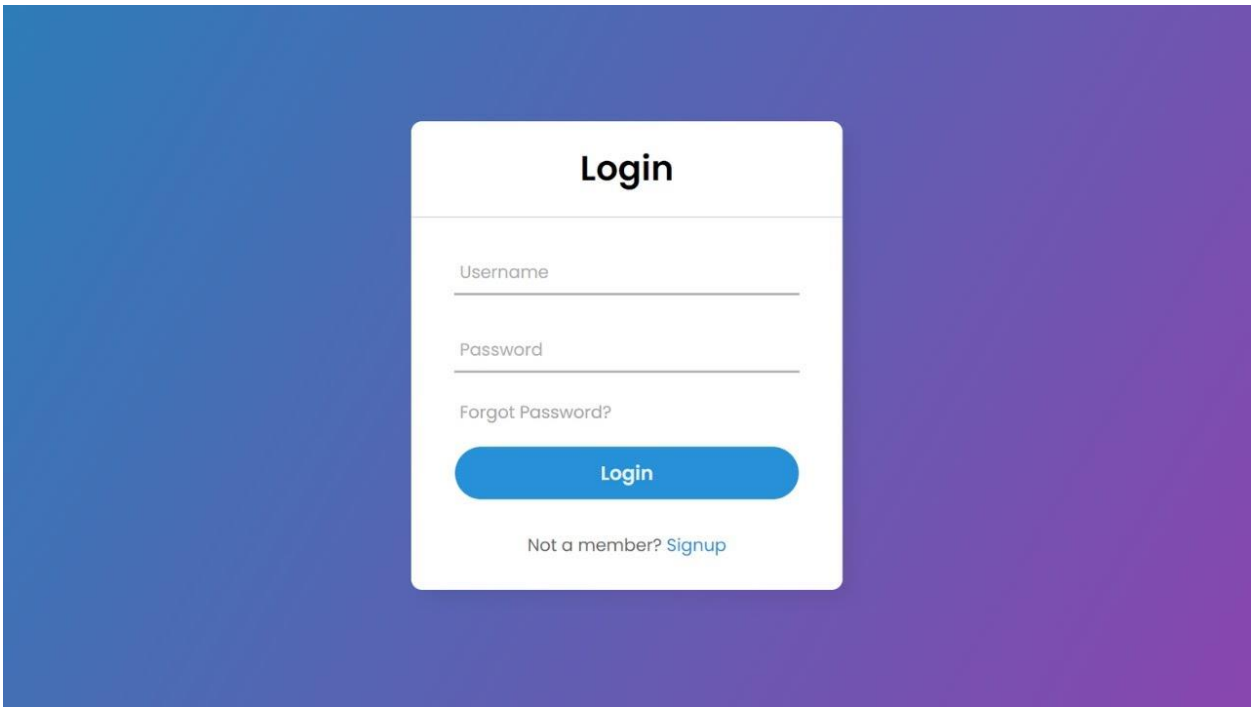
Örnek User API'leri

Kullanıcıların projelerine ayrılan şemalar altında tablolar oluşturulur ve içerisine veriler kaydedilebilir. Endpoint servis ile kullanıcılar bu tablolarına erişebilmek için kendi API'lerini oluşturur. Bu endpointler public şemasında tutulur. Kullanıcı bu endpointlere istek atarken projesi için özel oluşturulan apiKey değerini vermek zorundadır.

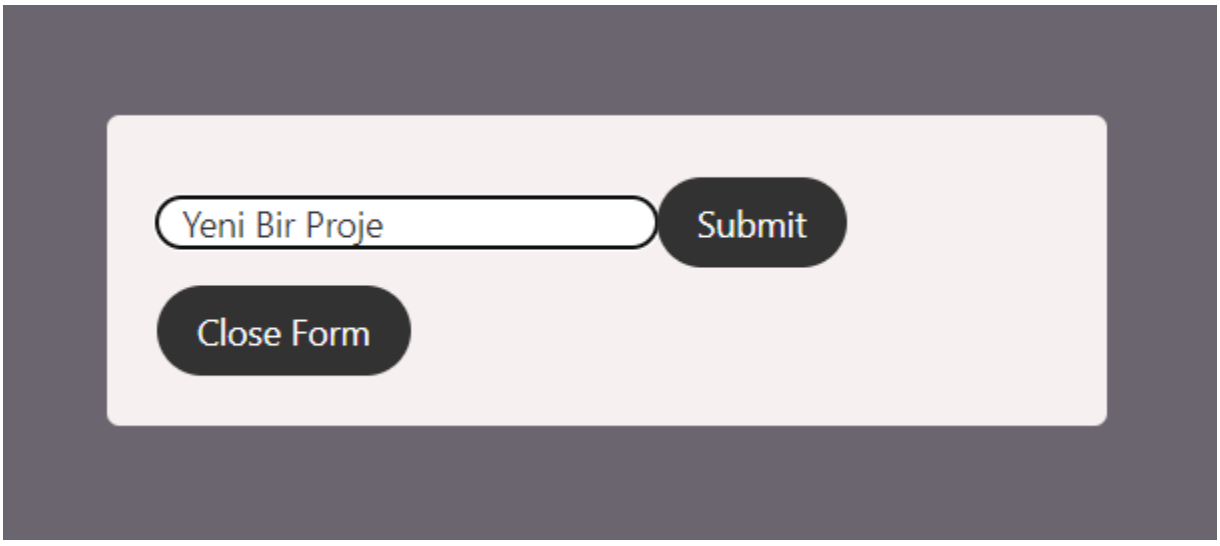
Tüm bu kurguları koda döktükten sonra görsel kısım olan Frontend için çalışmaya başladık. Temel düzeyde tasarımlar çıkardık ve koda döktük. Servisleri ekranlara bağlayarak isteklerimizi interaktif olarak test etmeye başladık. Frontend için planladığımız gibi bir React uygulamasını Visual Studio üzerinde oluşturduk. Gerekli bağımlılıkları ve konfigürasyonları yaptıktan sonra paket yapısını oluşturduk. Uygulamayı ayağa kaldırarak ilk testi gerçekleştirdik.



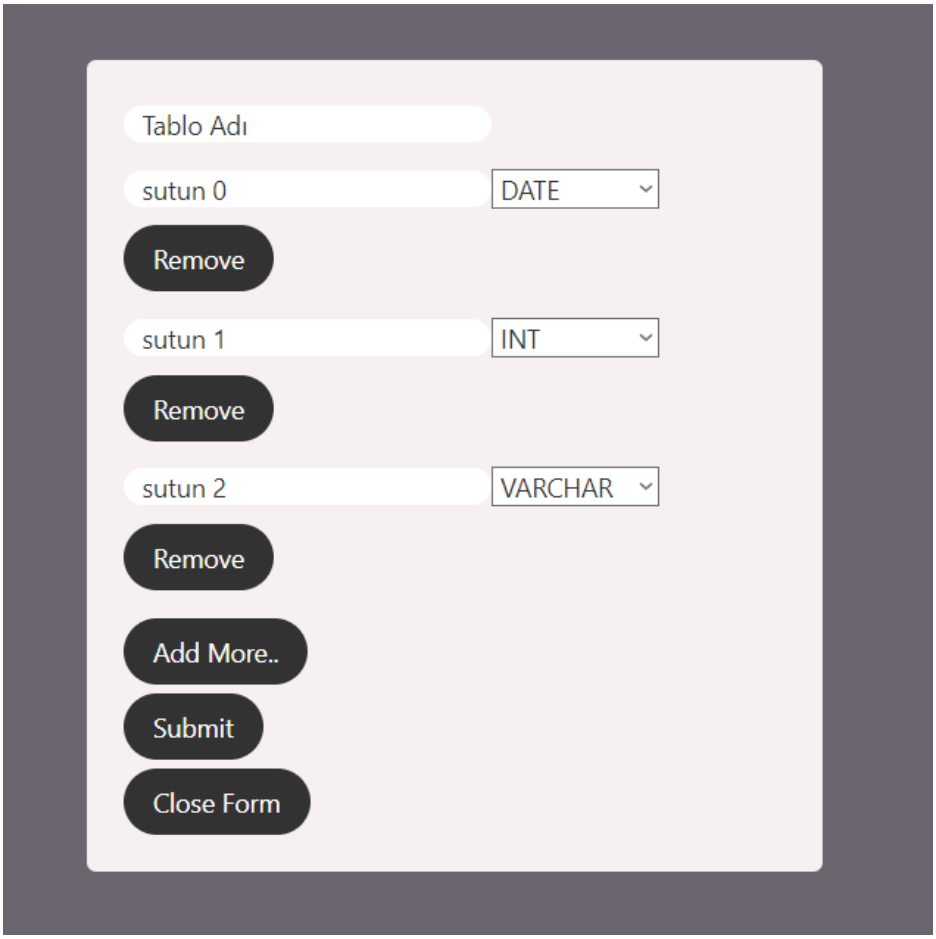
Sonrasında sırasıyla gerekli ekranların tasarımını kodlamaya başladık. İlk olarak uygulamanın giriş kısmı olan Login/Register ekranlarını tasarladık. Giriş yapmak için e-posta ve şifre bilgileri aldık. Ekranlardan aldığımız bilgileri ilgili servisimize bağlayarak doğrulama işlemlerini yaptık. Başarılı giriş yapan kullanıcılar için ana sayfanın tasarımına geçtik. Ana sayfada giriş yapan kullanıcılara ait genel bilgilerin yanı sıra oluşturdukları projeler listelenecek.



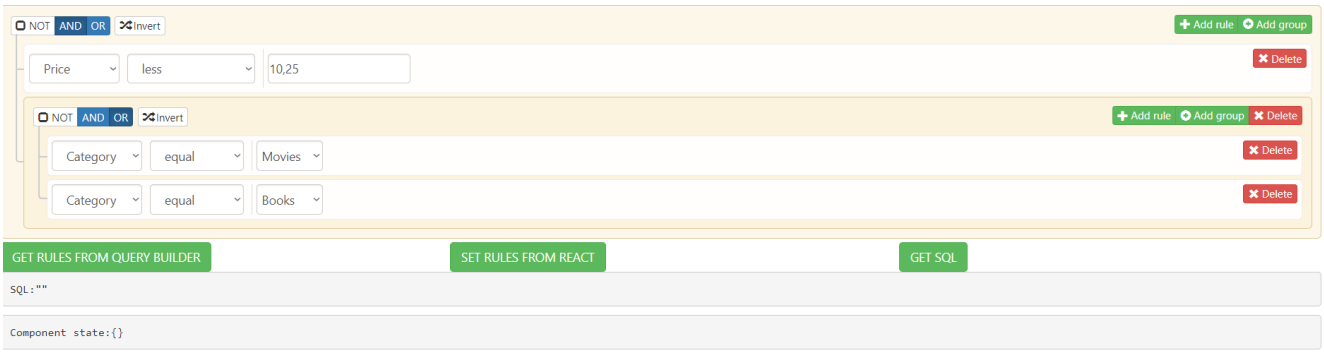
Ana sayfada kullanıcılar istedikleri projeye tıklayarak içeriğine ulaşabileceği gibi yeni bir projede başlatabilecekler. Bu doğrultuda geliştirdiğimiz tasarımı kodlamaya geçtik. Yeni proje oluşturmak için bir form üzerinden aldığımız bilgiler ile birlikte oturum açmış kullanıcının bilgilerini de kullanarak servisimizi çağırdık. Oluşturulan proje diğer projelerin yanına listelendi. Proje üzerine tıklayınca yeni bir sayfada projenin içeriği görüntülenecek.



Detay sayfasında kullanıcılar seçtikleri projeye ait tabloların listesine ulaşabileceği gibi yeni bir tablo da oluşturabilecekler. Aynı zamanda endpointlerin bir listesini ve yeni endpoint oluşturma butonu da görünecektir. Bu doğrultuda geliştirdiğimiz tasarımı kodlamaya geçtik. Yeni tablo oluşturmak için bir form üzerinden aldığımız bilgiler ile birlikte seçilen projeye ait bilgileri de kullanarak servisimizi çağırdık. Oluşturulan tablo diğer tabloların yanına listelendi. Tablo üzerine tıklayınca yeni bir sayfada tablonun içeriğindeki veriler görüntülenecek.



Yeni bir endpoint oluşturmak için oluşturduğumuz sayfada bulunan QueryBuilder yapısı ile tabloların ve alanlarının bilgileri yer almaktadır. jQuery-QueryBuilder eklentisini kullanarak bir sorgu oluşturucu sağlar. Sorgu oluşturucusu, kullanıcılara farklı filtrelerin ve kuralların kullanıldığı bir arayüz sunar. Kullanıcılar filtreleri seçebilir, kurallar oluşturabilir ve sorguyu oluşturabilir. "GET RULES FROM QUERY BUILDER" düğmesi, oluşturulan kuralları alarak ve "SET RULES FROM REACT" düğmesi, kuralları React bileşeninden ayarlayarak işlevselliği gösterir. "GET SQL" düğmesi, oluşturulan SQL sorgusunu gösterir. Bileşenin durumu ve oluşturulan SQL sorgusu da gösterilir. Kullanıcı almak istediği verileri ve değerlerini belirledikten sonra endpointi oluşturabilmektedir. Oluşturdukları bu servise projelerine ait apiKey bilgisi ile istekte bulunmaları durumunda istedikleri verilere erişebileceklerdir.



Tamamlanan ekran tasarımlarının ardından uygulamanın eksik yönleri ve geliştirilmesi gereken özellikler hakkında incelemeler yapıp projeyi tamamladık.

6. Kaynakça

https://link.springer.com/chapter/10.1007/978-3-031-16947-2_1
https://www.diva-portal.org/smash/get/diva2:1677217/FULLTEXT01.pdf
https://arxiv.org/ftp/arxiv/papers/2205/2205.04212.pdf
https://avesis.gazi.edu.tr/resume/downloadfile/firat?key=c09290f8-3e06-43c7-9dad-43912528fc1d
https://avesis.gazi.edu.tr/yayin/27bce3f0-e9ba-4044-9f44-2eac91b4ce17/design-and-implementation-of-a-web-based-intrusion-prevention-system-a-new-hybrid-model
https://dergipark.org.tr/en/download/article-file/1205971
https://dergipark.org.tr/en/download/article-file/697402
https://dergipark.org.tr/en/download/article-file/323276
https://wmaraci.com/nedir/postgresql
https://www.alastyr.com/blog/react-nedir-react-nasil-calisir/
https://www.argenova.com.tr/react-js-nedir
https://tr.wikipedia.org/wiki/JavaScript
https://aws.amazon.com/tr/what-is/javascript/
https://bulutistan.com/blog/veri-tabani-yonetim-sistemi-nedir/